

Chapter 5: Backpropagation

Basics and Applications of Deep Learning

Fall 2026

Department of Mathematical Data Science

Hanyang University ERICA

Lecturer: Sehyun Kwon



(Recap) what are we actually trying to learn?

Choosing **weights (W)** that makes the loss small



(Recap) what are we actually trying to learn?

Choosing **weights (W)** that makes the loss small

However, for deep neural networks, we cannot directly compute the exact minimizer in practice.

(Recap) what are we actually trying to learn?

Choosing **weights (W)** that makes the loss small

However, for deep neural networks, we cannot directly compute the exact minimizer in practice.

Why? Deep neural networks compose millions or billions of nonlinear operations.

(Recap) what are we actually trying to learn?

Choosing **weights (W)** that makes the loss small

However, for deep neural networks, we cannot directly compute the exact minimizer in practice.

Why? Deep neural networks compose millions or billions of nonlinear operations.

Therefore, no mathematical formula tells us the best **W** in **one shot**.

(Recap) solution: move the weights a little at a time

Instead of exactly solving for the answer, repeatedly take a step that reduces the loss.

(Recap) solution: move the weights a little at a time

Instead of exactly solving for the answer, repeatedly take a step that reduces the loss.

How?

(Recap) solution: move the weights a little at a time

Instead of exactly solving for the answer, repeatedly take a step that reduces the loss.

How? Gradient descent!

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

← Loss

(Recap) solution: move the weights a little at a time

Instead of exactly solving for the answer, repeatedly take a step that reduces the loss.

How? Gradient descent!

$$W \leftarrow W - \eta \frac{\partial L}{\partial W} \quad \leftarrow \text{Loss}$$

Now all we need is the gradient. How can we compute it?

How can we compute $\frac{\partial L}{\partial W}$?

(Bad idea) Compute it by hand: **It does NOT scale**. There are countless possible loss functions and model architectures.

How can we compute $\frac{\partial L}{\partial W}$?

(Bad idea) Compute it by hand: **It does NOT scale**. There are countless possible loss functions and model architectures.

(Better idea) A deep model already has a **computational flow**!

How can we compute $\frac{\partial L}{\partial W}$?

(Bad idea) Compute it by hand: **It does NOT scale**. There are countless possible loss functions and model architectures.

(Better idea) **A deep model already has a computational flow!**

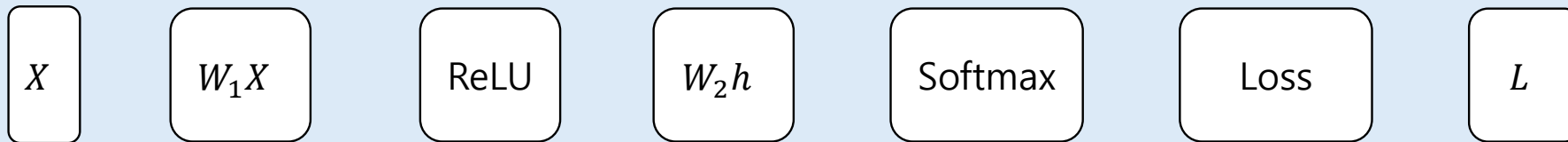
$$\text{e.g., } L = \text{Loss}(\text{softmax}(W_2 \text{ReLU}(W_1 x)), y)$$

How can we compute $\frac{\partial L}{\partial W}$?

(Bad idea) Compute it by hand: **It does NOT scale**. There are countless possible loss functions and model architectures.

(Better idea) **A deep model already has a computational flow!**

e.g., $L = \text{Loss}(\text{softmax}(W_2 \text{ReLU}(W_1 x)), y)$



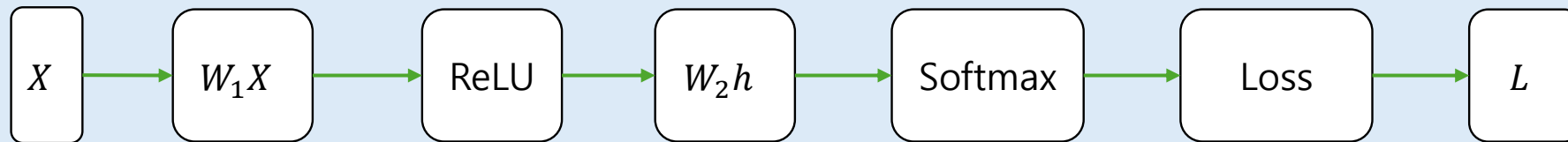
How can we compute $\frac{\partial L}{\partial W}$?

(Bad idea) Compute it by hand: **It does NOT scale**. There are countless possible loss functions and model architectures.

(Better idea) **A deep model already has a computational flow!**

e.g., $L = \text{Loss}(\text{softmax}(W_2 \text{ReLU}(W_1 x)), y)$

forward



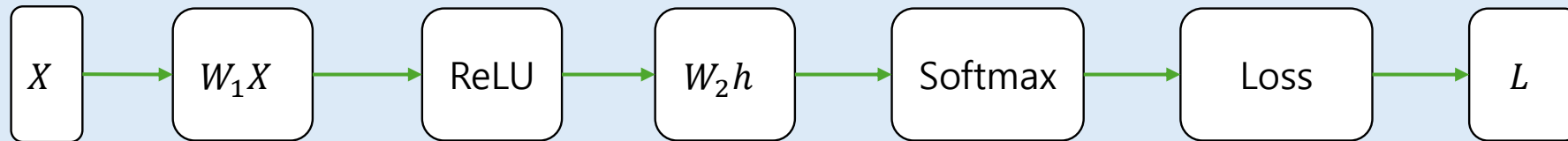
How can we compute $\frac{\partial L}{\partial W}$?

(Bad idea) Compute it by hand: **It does NOT scale**. There are countless possible loss functions and model architectures.

(Better idea) **A deep model already has a computational flow!**

$$\text{e.g., } L = \text{Loss}(\text{softmax}(W_2 \text{ReLU}(W_1 x)), y)$$

forward



Gradients flow backward from the loss through the computational graph via the **chain rule**.

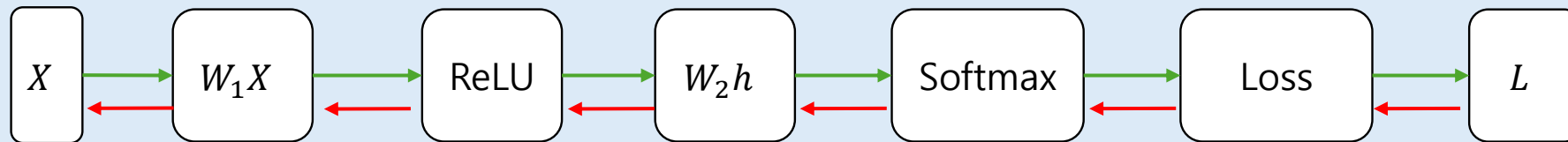
How can we compute $\frac{\partial L}{\partial W}$?

(Bad idea) Compute it by hand: **It does NOT scale**. There are countless possible loss functions and model architectures.

(Better idea) **A deep model already has a computational flow!**

e.g., $L = \text{Loss}(\text{softmax}(W_2 \text{ReLU}(W_1 x)), y)$

forward



backward

Gradients flow backward from the loss through the computational graph via the **chain rule**.

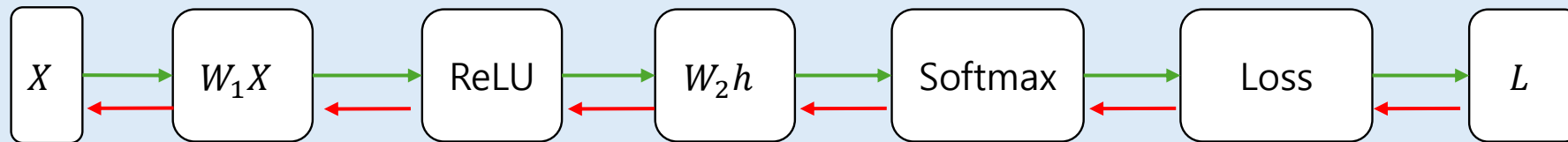
How can we compute $\frac{\partial L}{\partial W}$?

(Bad idea) Compute it by hand: **It does NOT scale**. There are countless possible loss functions and model architectures.

(Better idea) A deep model already has a **computational flow**!

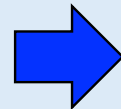
$$\text{e.g., } L = \text{Loss}(\text{softmax}(W_2 \text{ReLU}(W_1 x)), y)$$

forward



backward

Gradients flow backward from the loss through the computational graph via the **chain rule**.



Today's topic

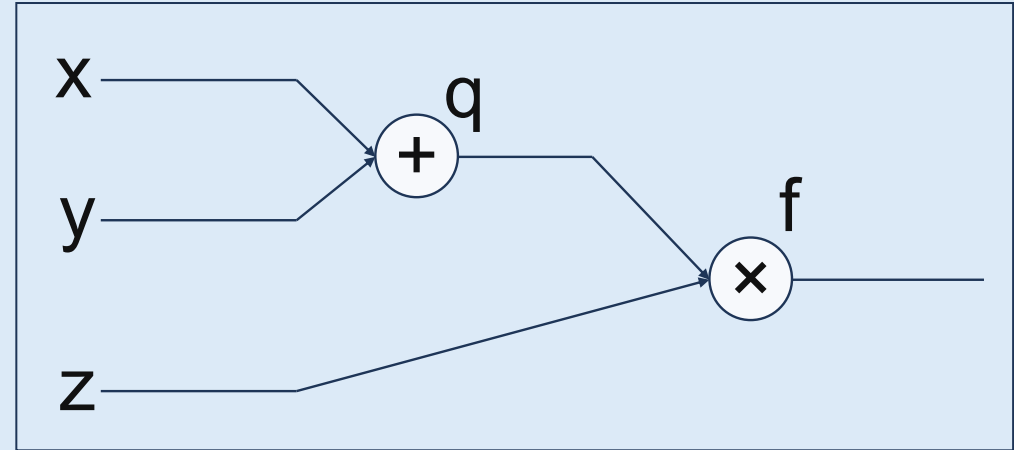
Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$



Backpropagation: a simple example

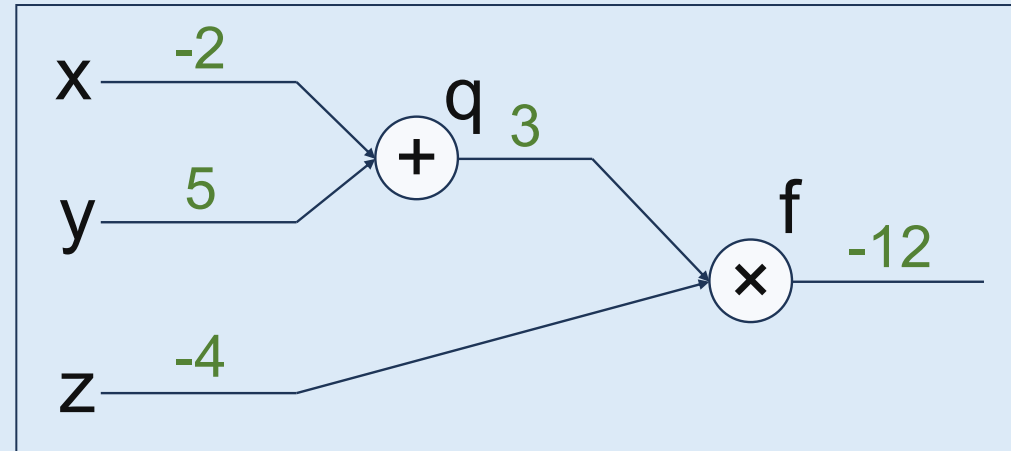
$$f(x, y, z) = (x + y)z$$



Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

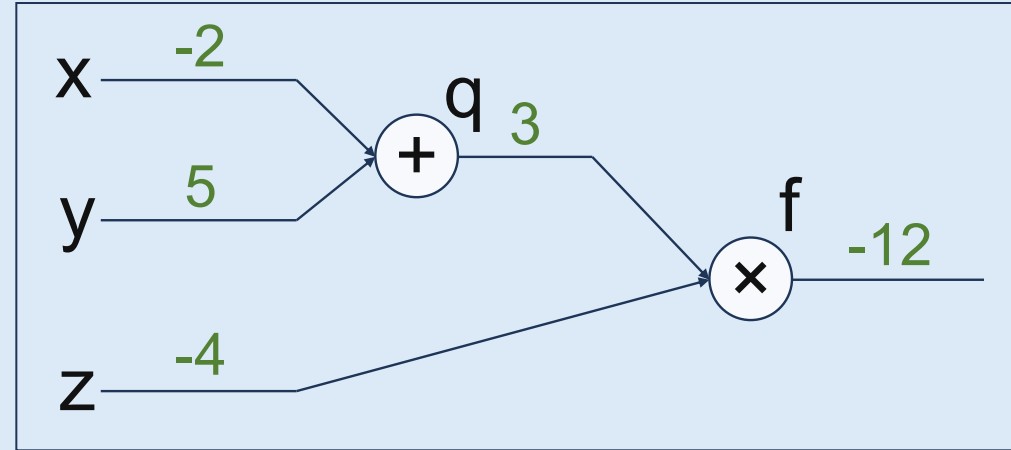


Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$



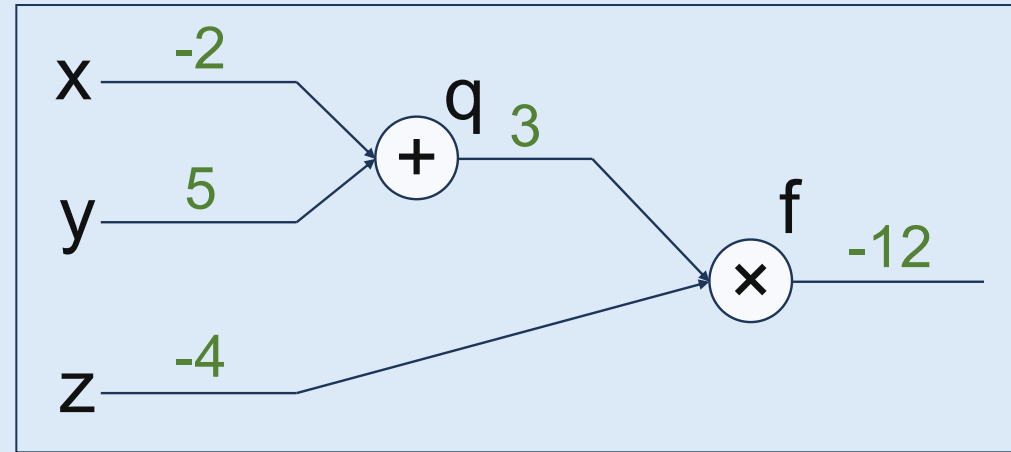
Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$



Backpropagation: a simple example

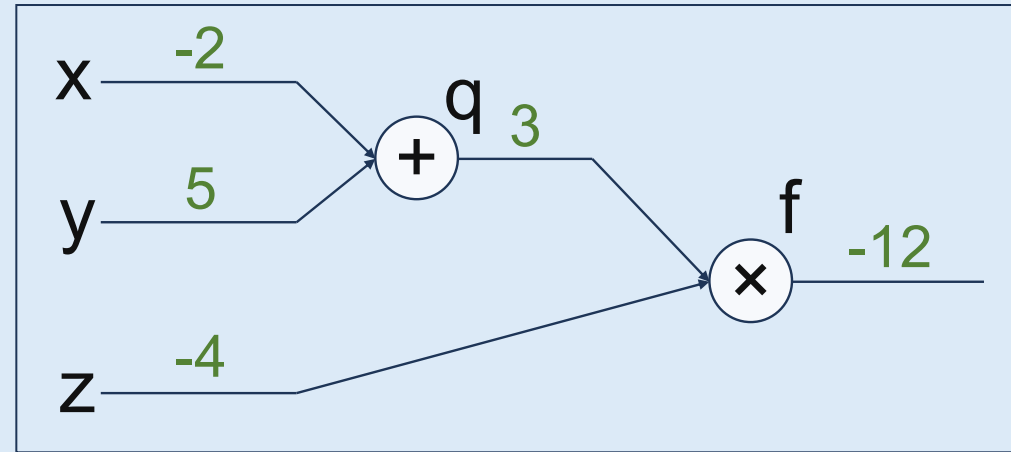
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation: a simple example

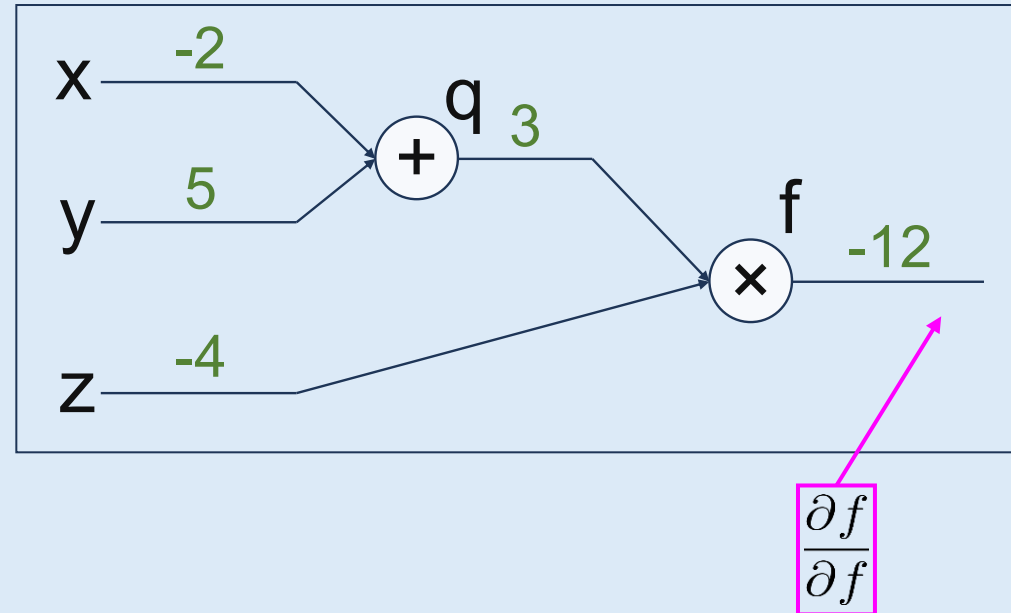
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation: a simple example

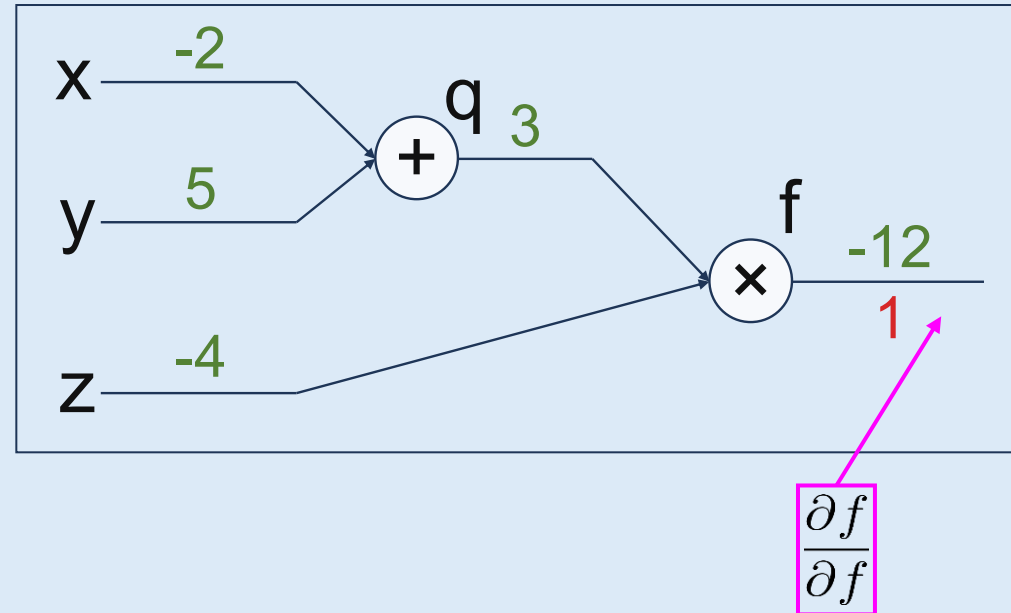
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation: a simple example

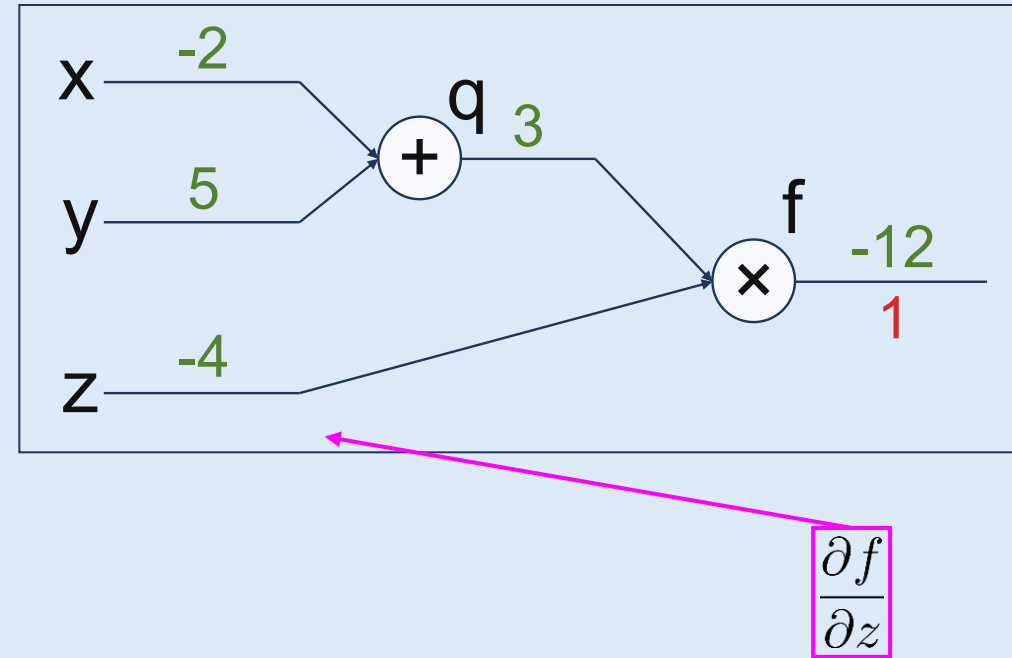
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation: a simple example

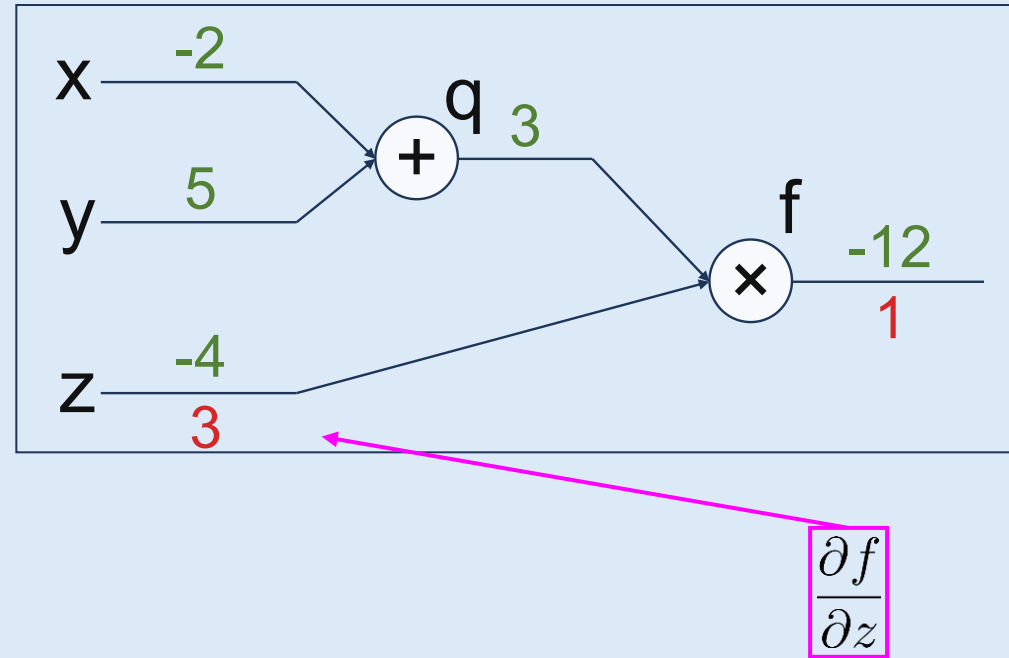
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation: a simple example

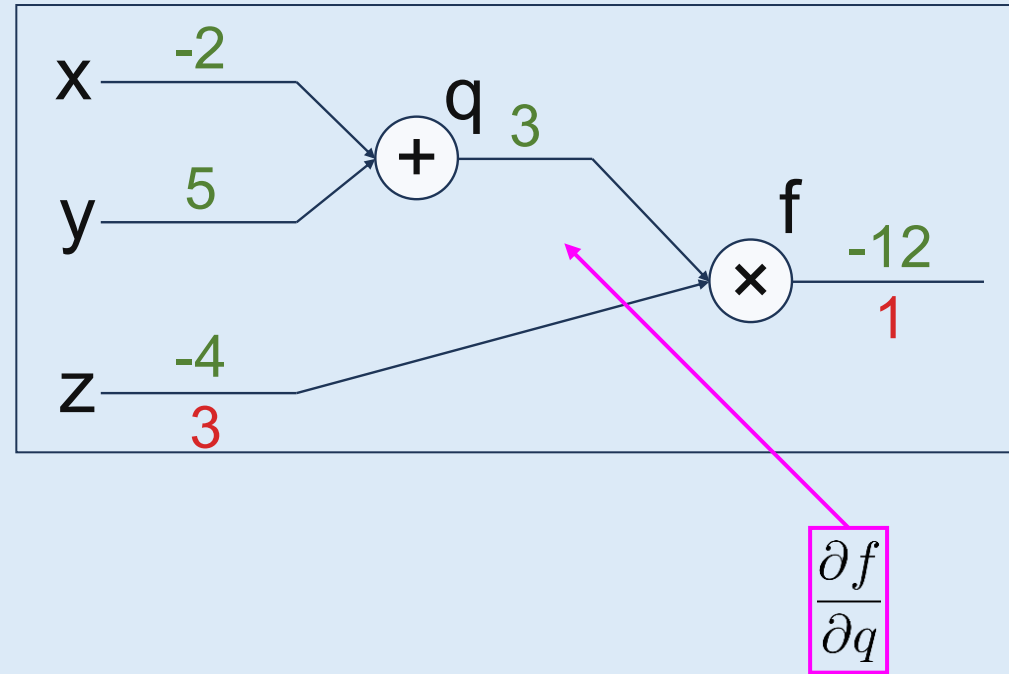
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation: a simple example

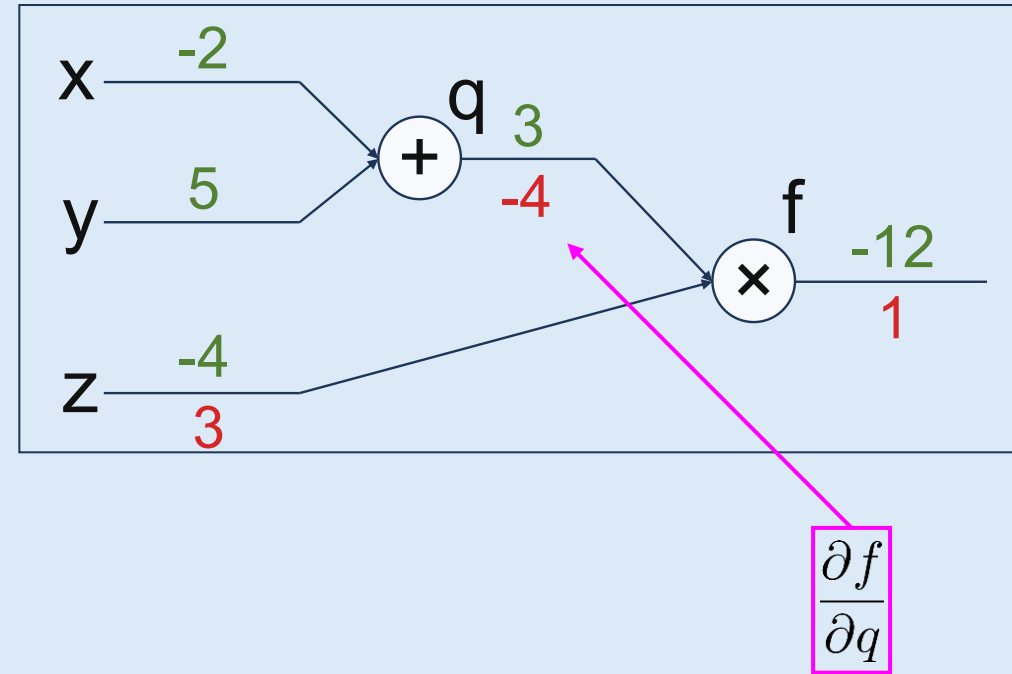
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation: a simple example

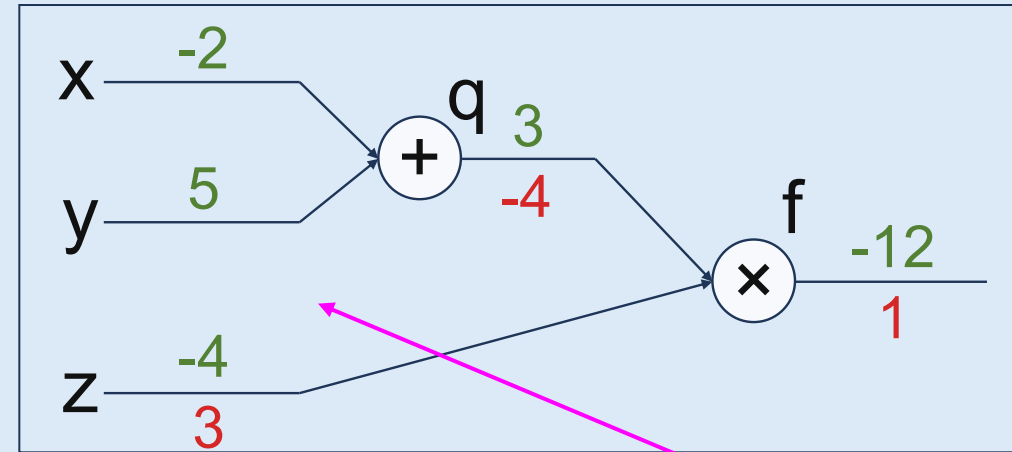
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Upstream
gradient

Local
gradient

$$\frac{\partial f}{\partial y}$$

Backpropagation: a simple example

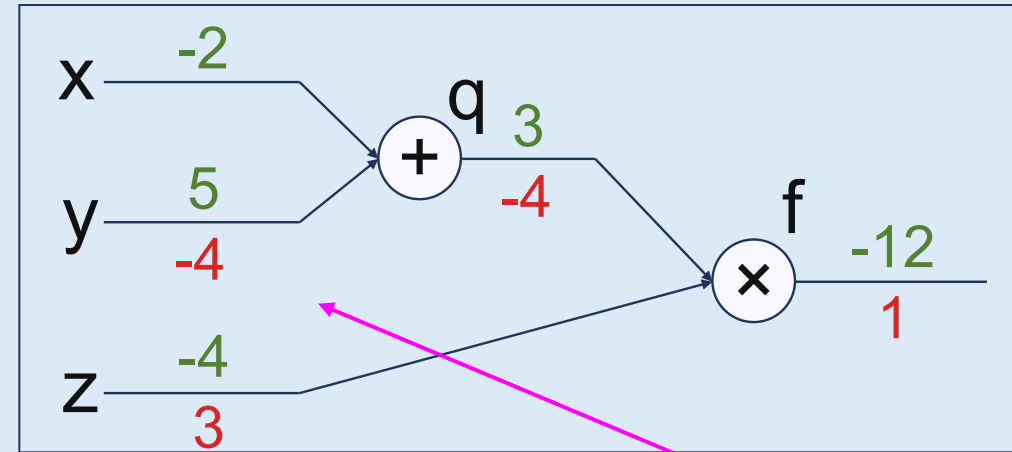
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Upstream
gradient

Local
gradient

$$\frac{\partial f}{\partial y}$$

Backpropagation: a simple example

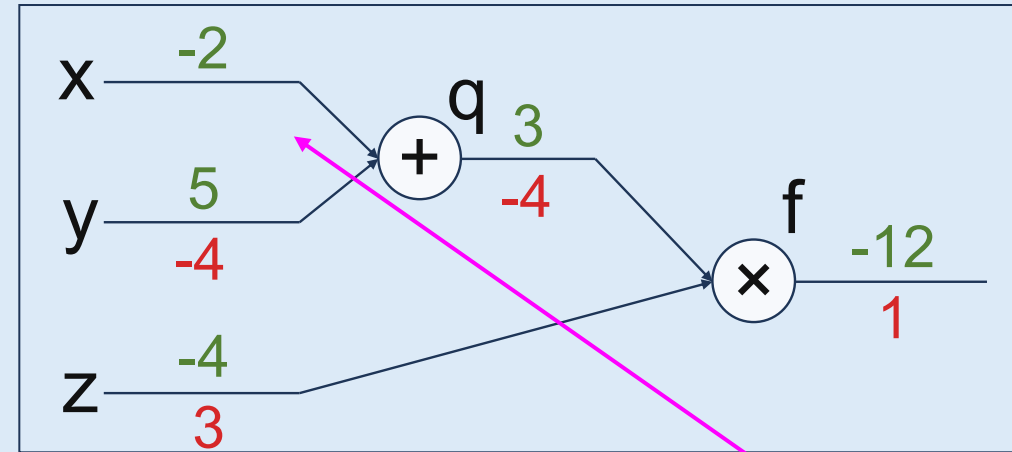
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

Upstream
gradient

Local
gradient

$$\frac{\partial f}{\partial x}$$

Backpropagation: a simple example

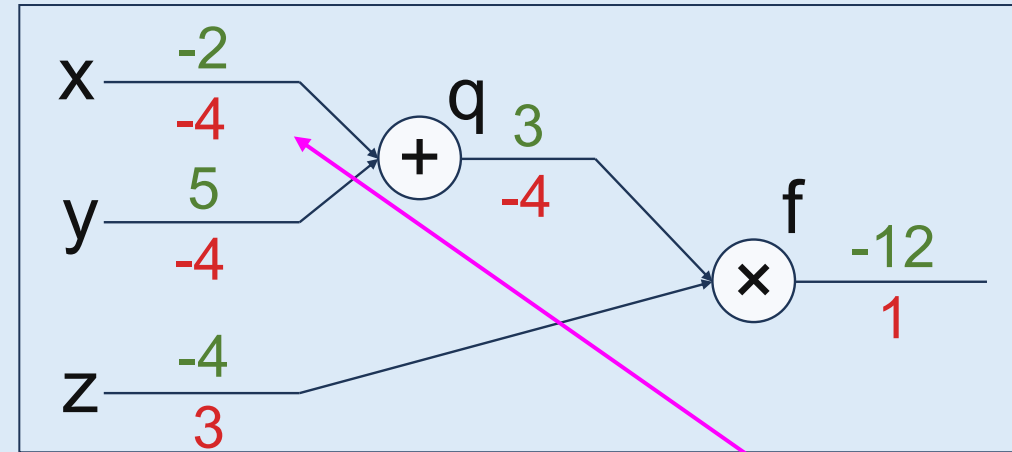
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



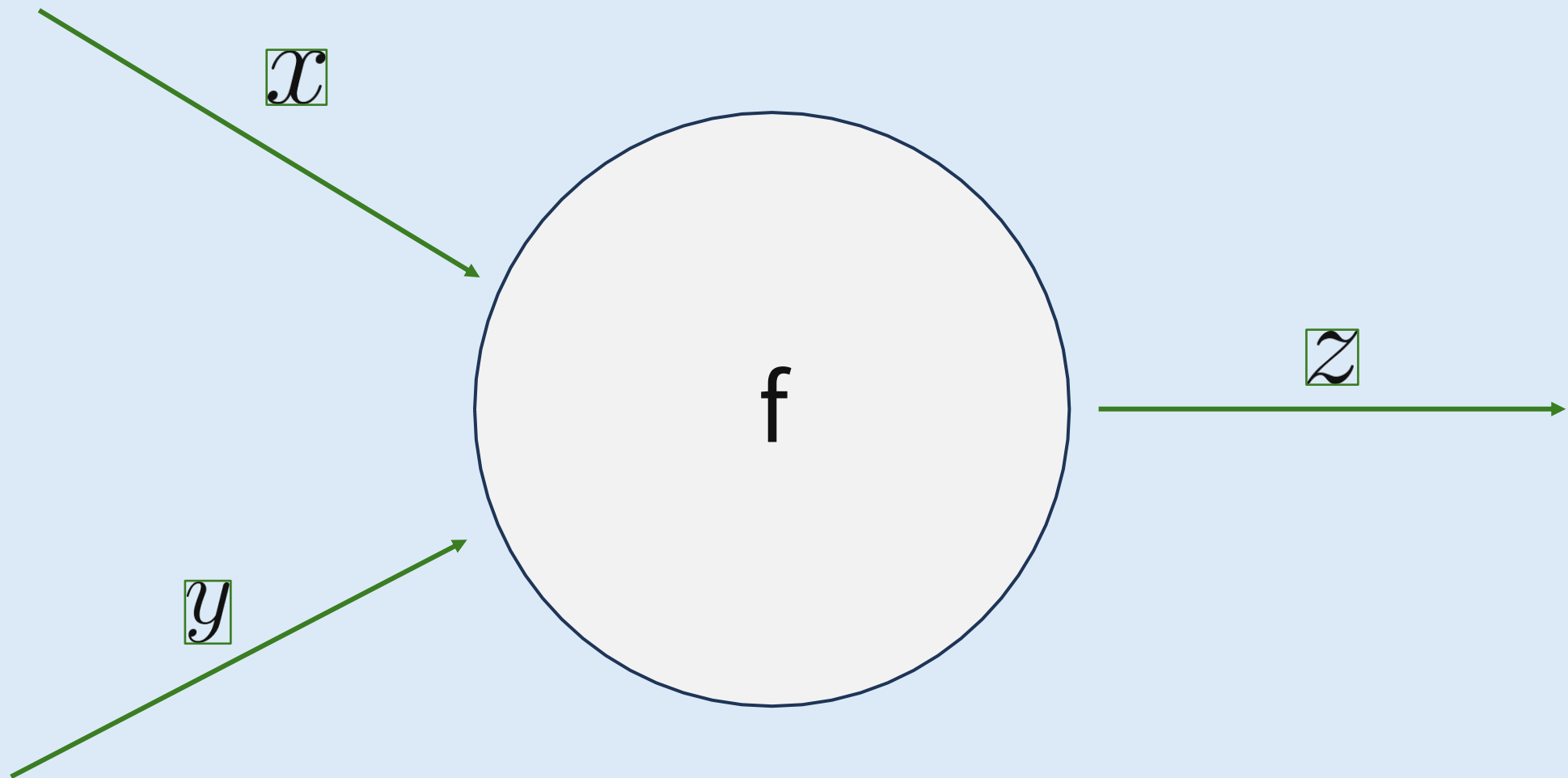
Chain rule:

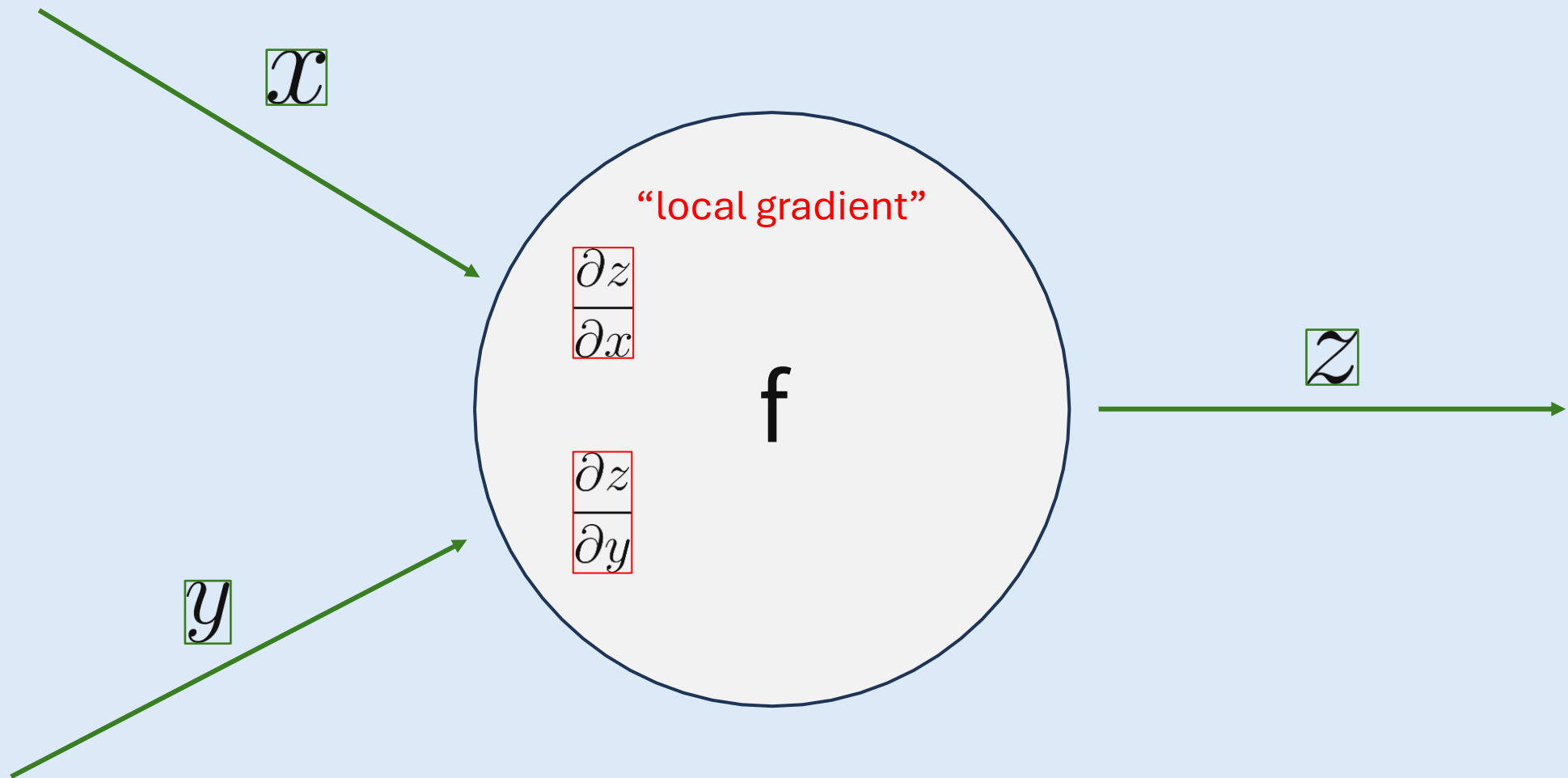
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

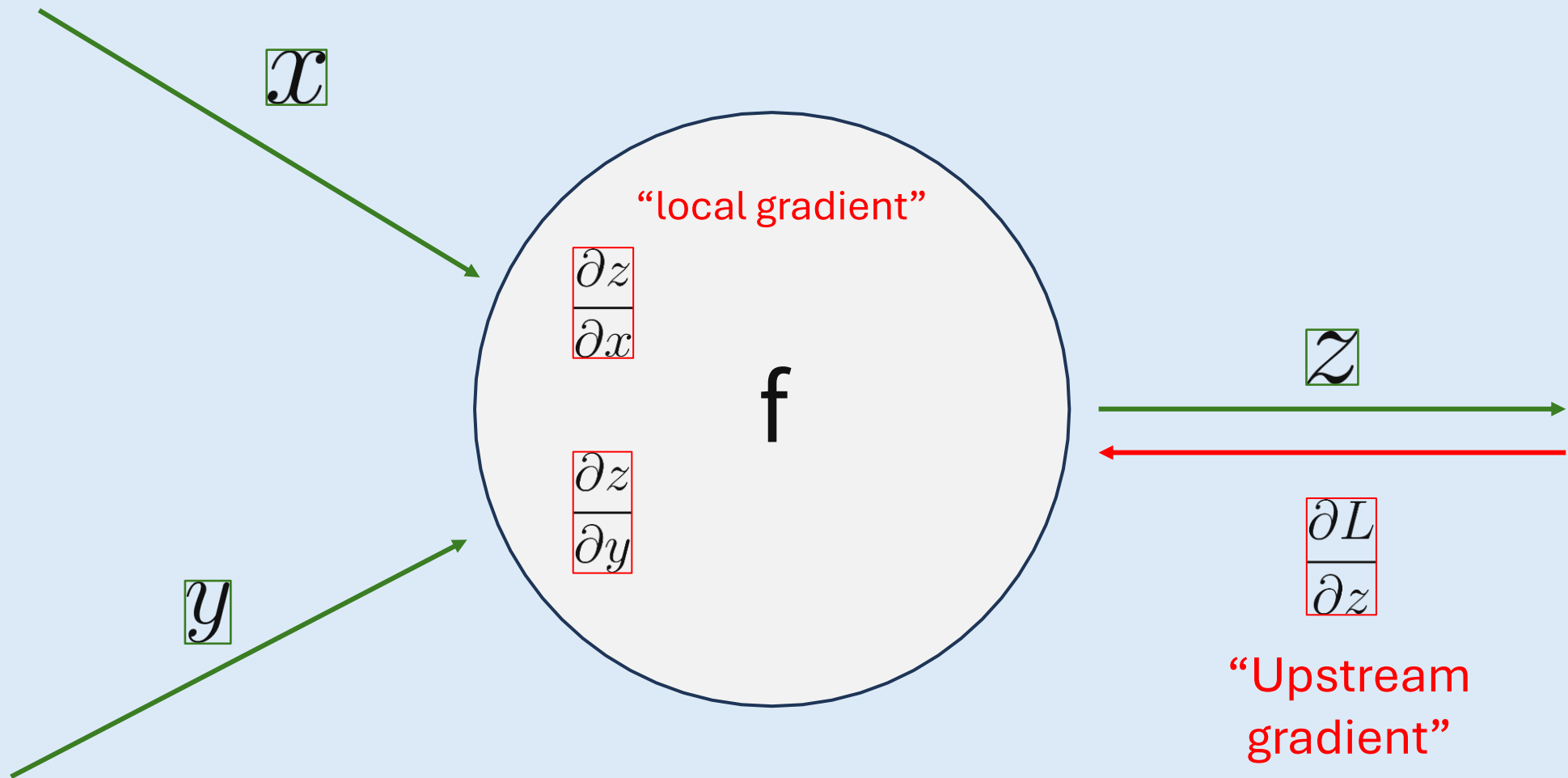
Upstream
gradient

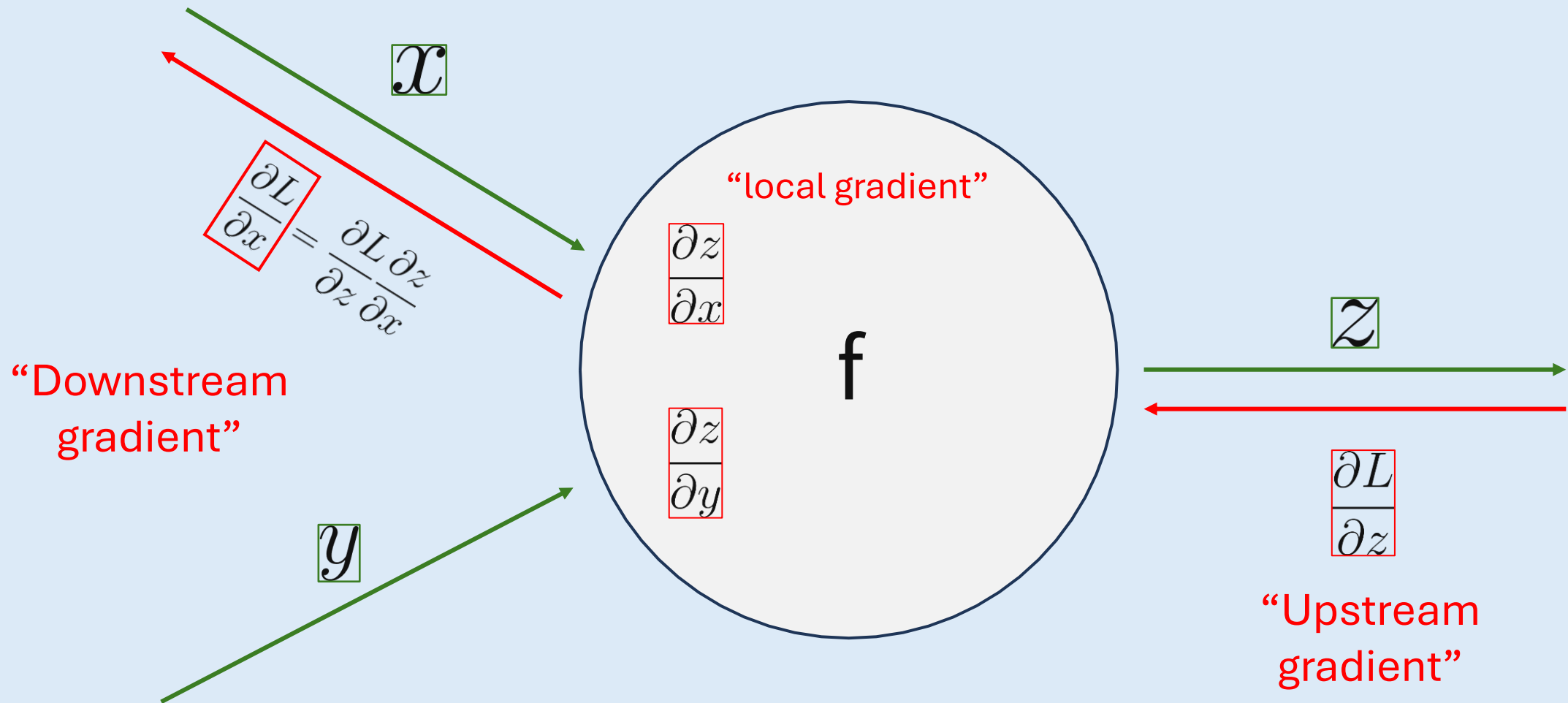
Local
gradient

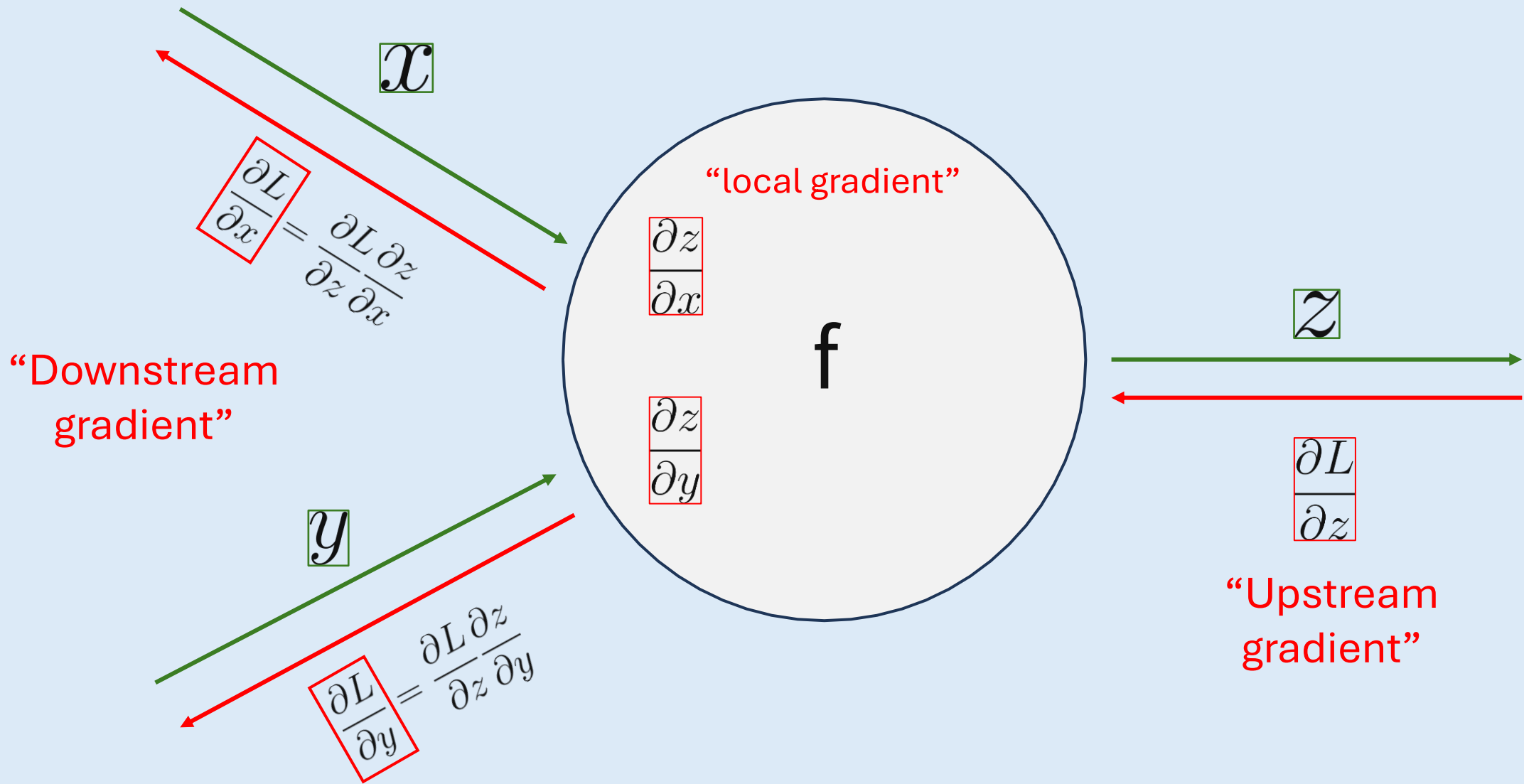
$$\frac{\partial f}{\partial x}$$

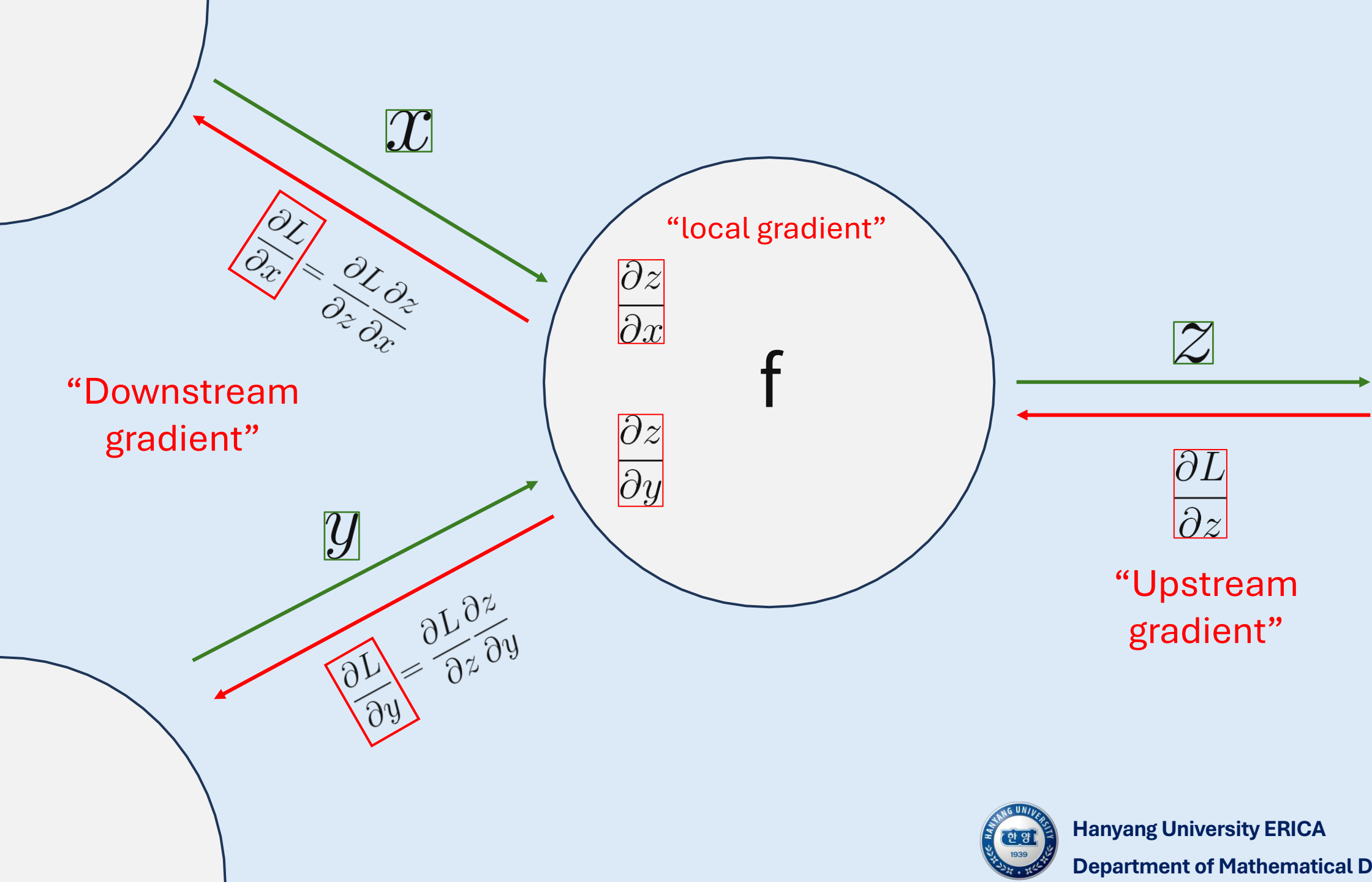




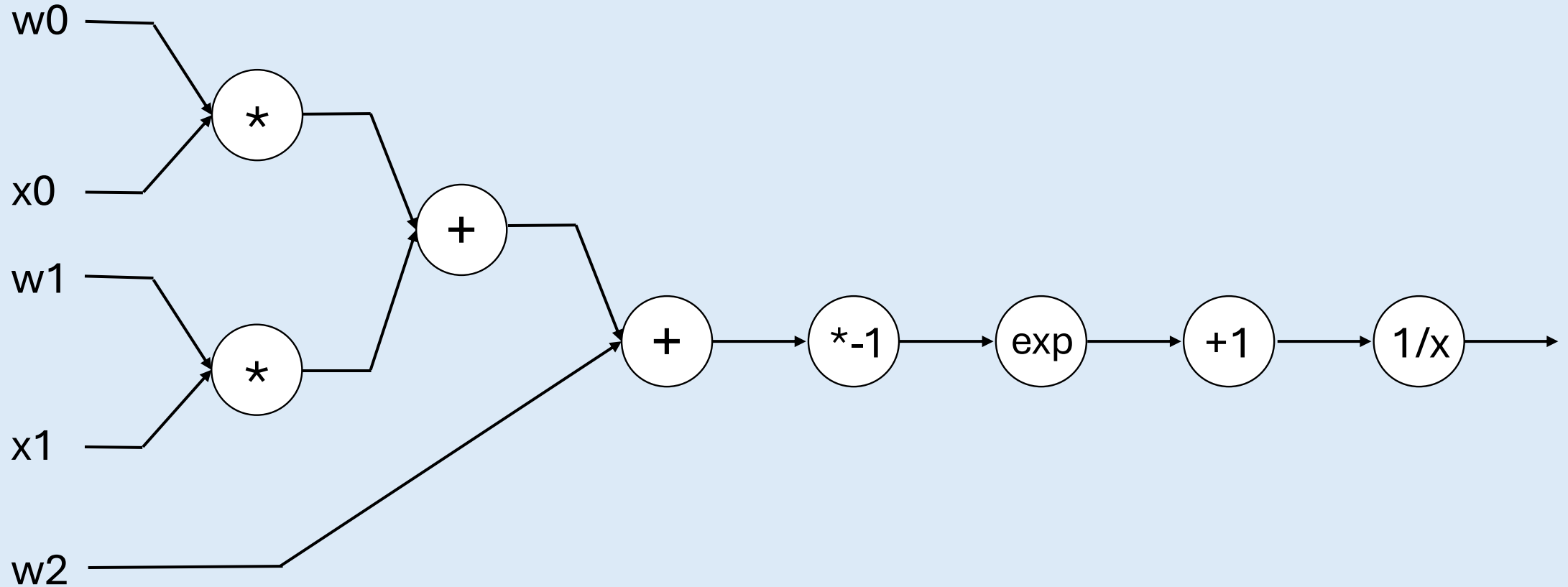




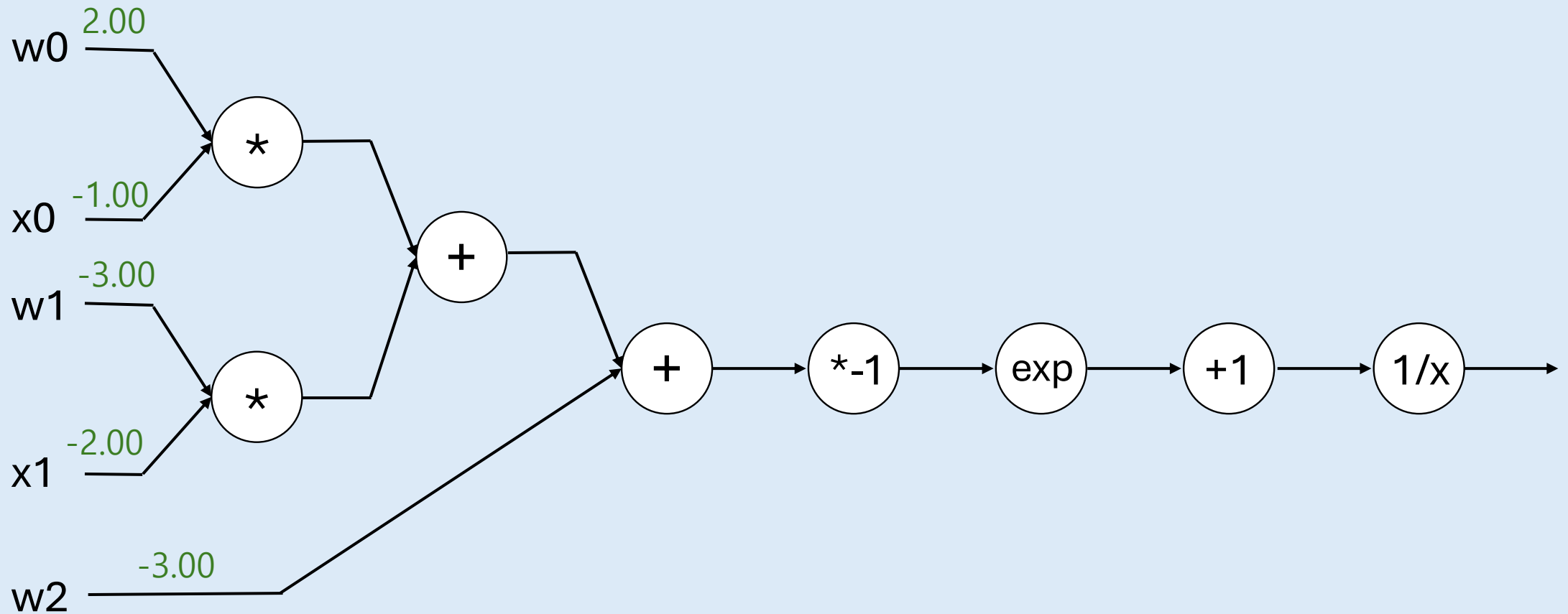




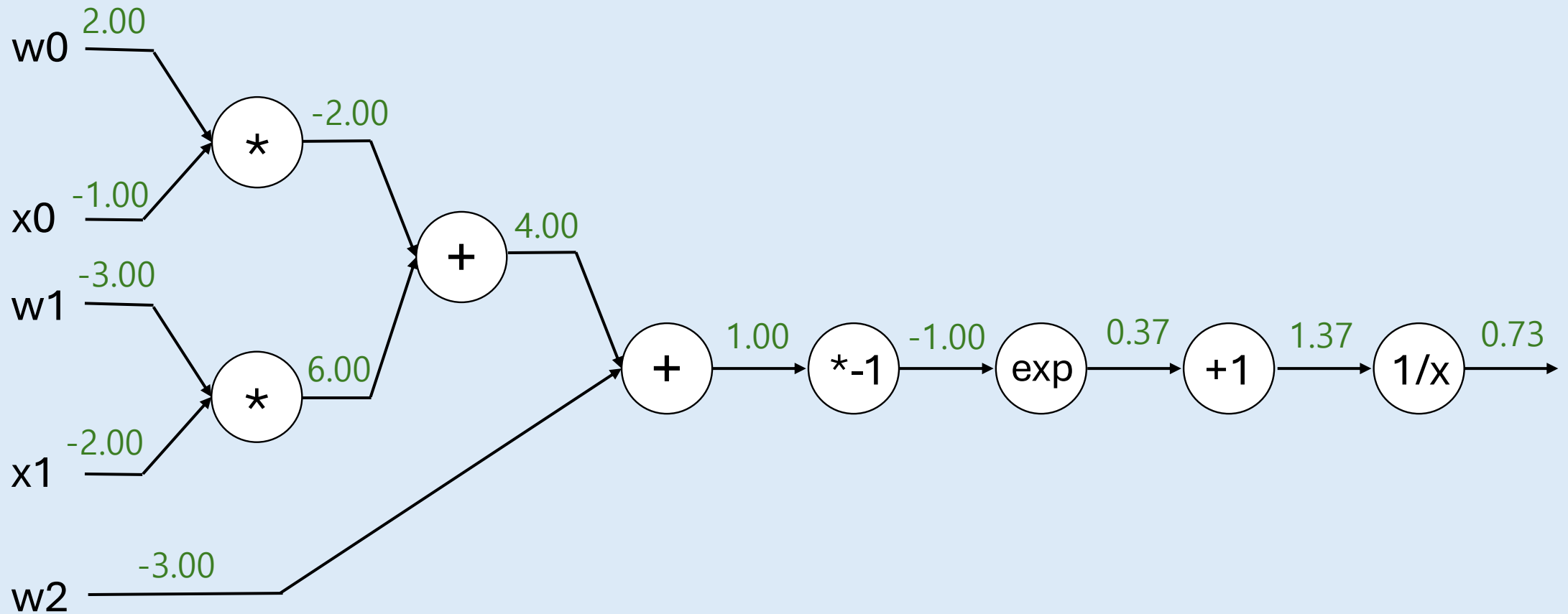
Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



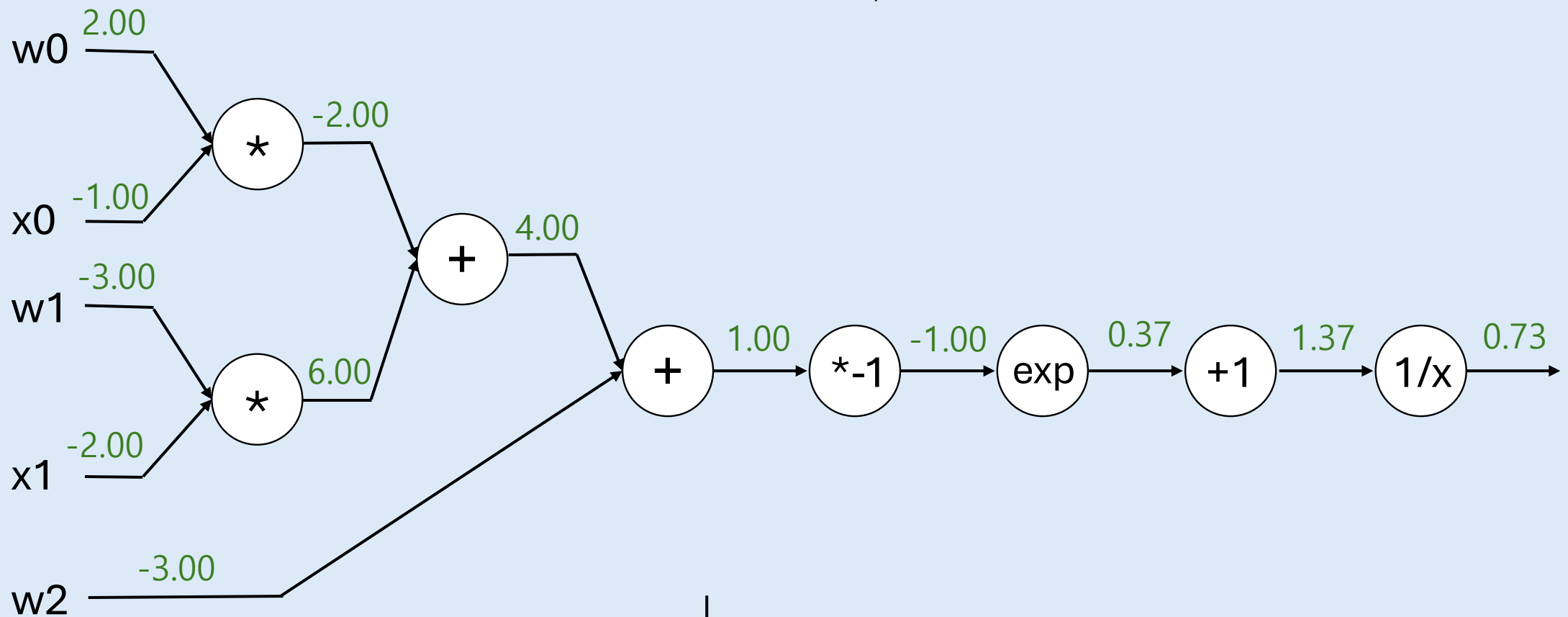
Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



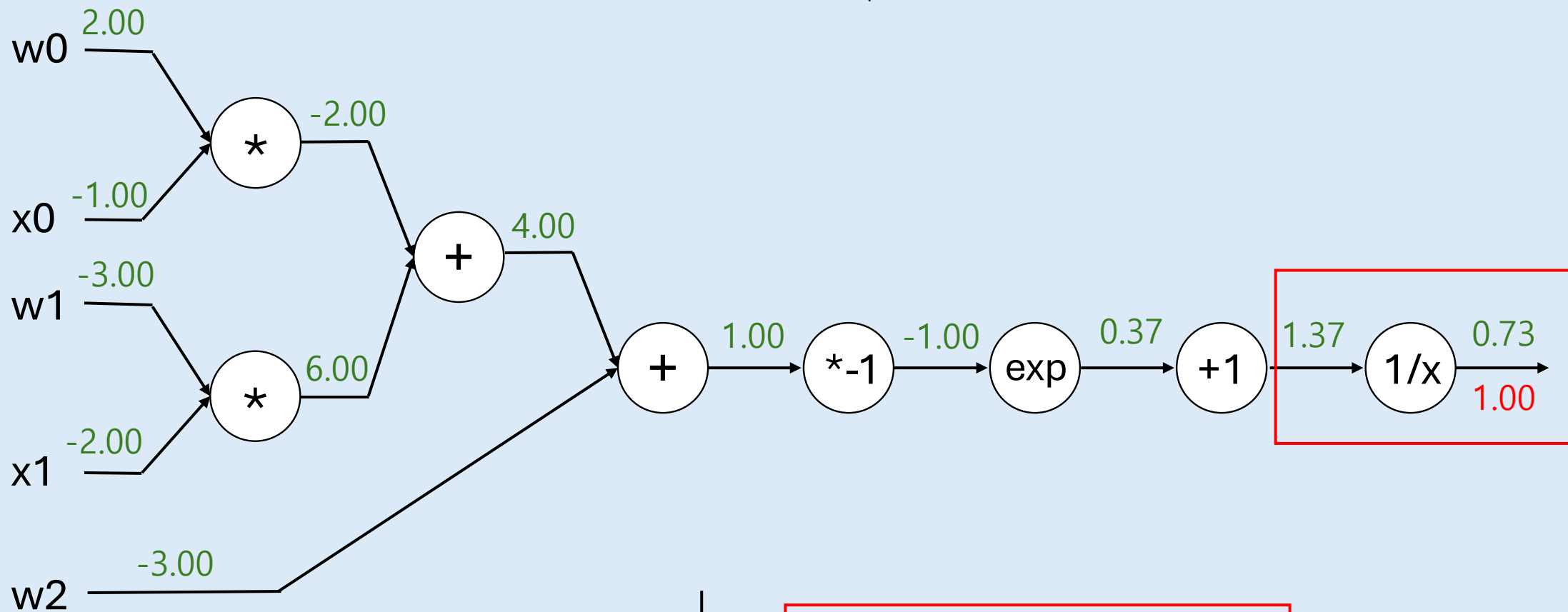
$$f(x) = e^x \longrightarrow \frac{df}{dx} = e^x$$

$$f_a(x) = ax \longrightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \longrightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \longrightarrow \frac{df}{dx} = 1$$

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



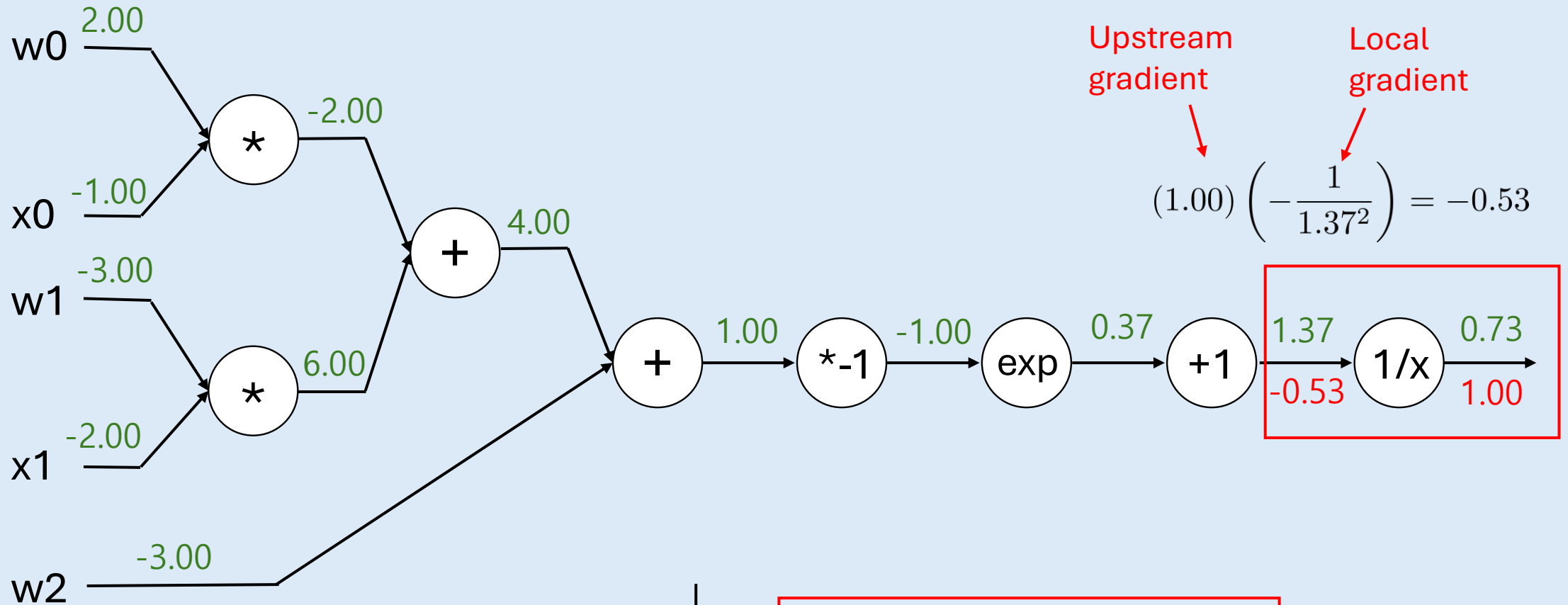
$$f(x) = e^x \longrightarrow \frac{df}{dx} = e^x$$

$$f_a(x) = ax \longrightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \longrightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \longrightarrow \frac{df}{dx} = 1$$

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



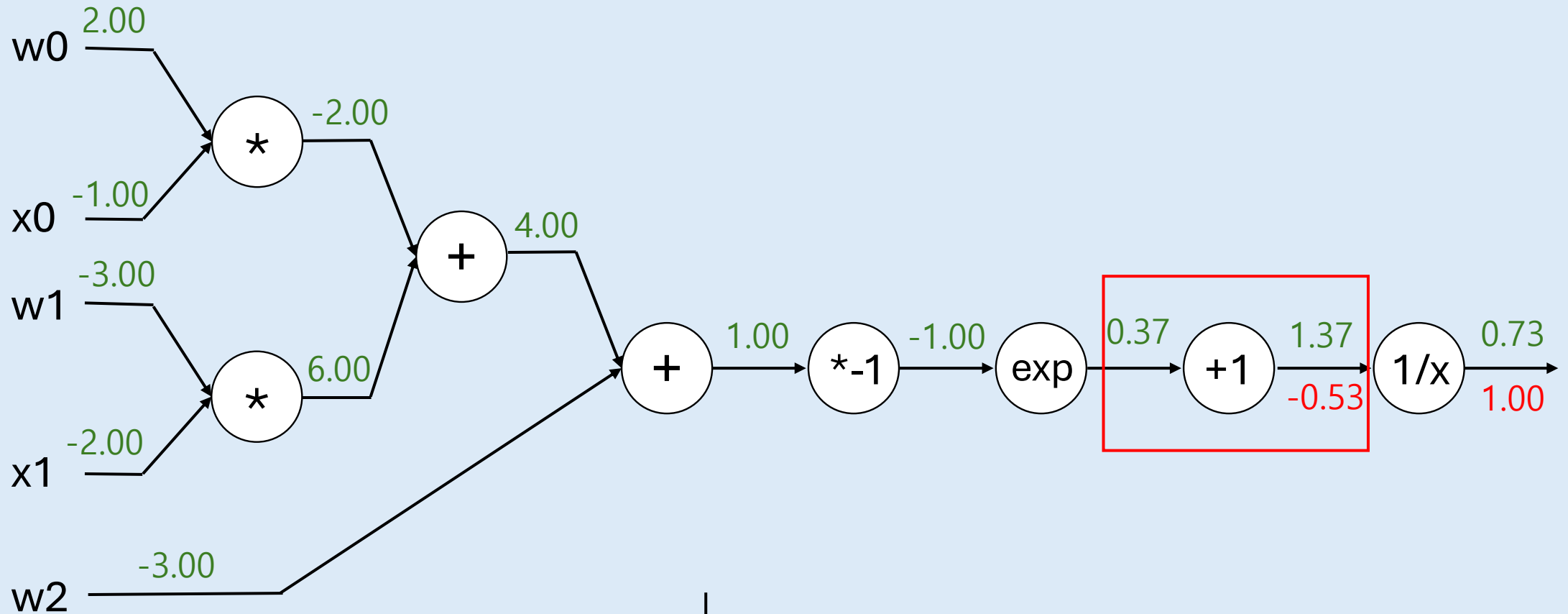
$$f(x) = e^x \longrightarrow \frac{df}{dx} = e^x$$

$$f_a(x) = ax \longrightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \longrightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \longrightarrow \frac{df}{dx} = 1$$

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



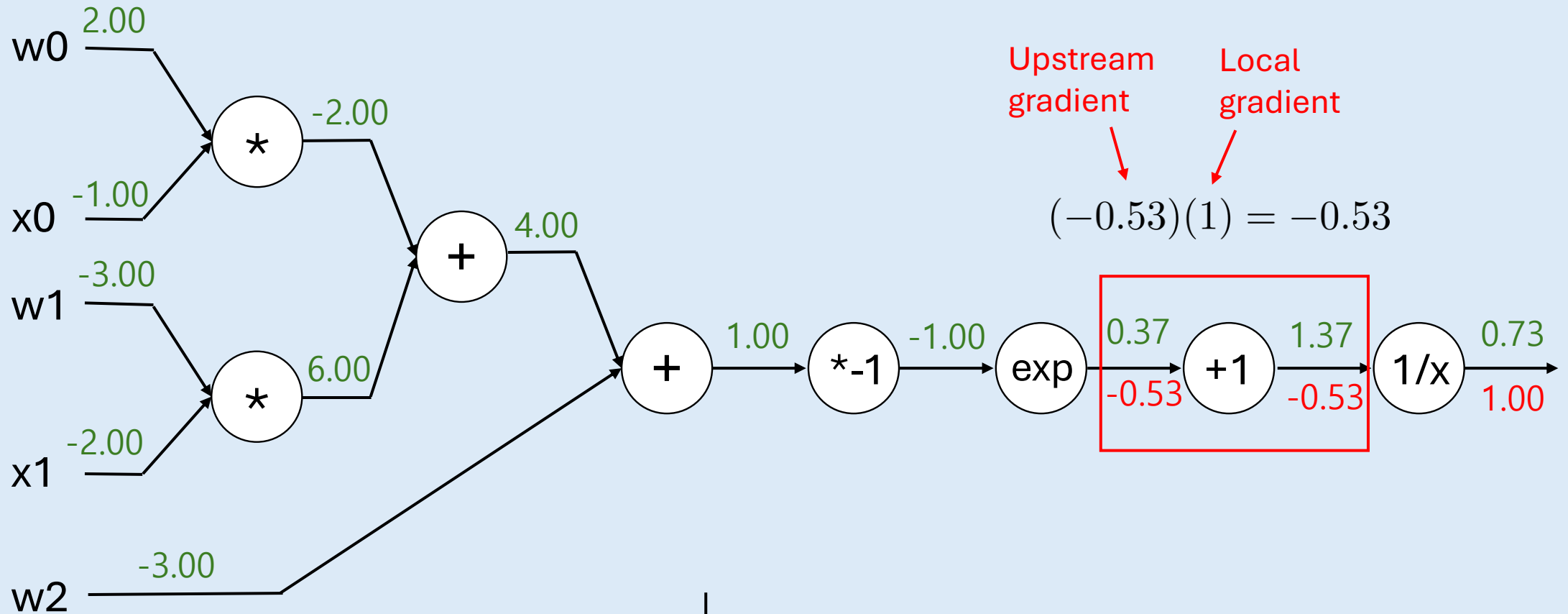
$$f(x) = e^x \longrightarrow \frac{df}{dx} = e^x$$

$$f_a(x) = ax \longrightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \longrightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \longrightarrow \frac{df}{dx} = 1$$

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



Upstream gradient
Local gradient
 $(-0.53)(1) = -0.53$

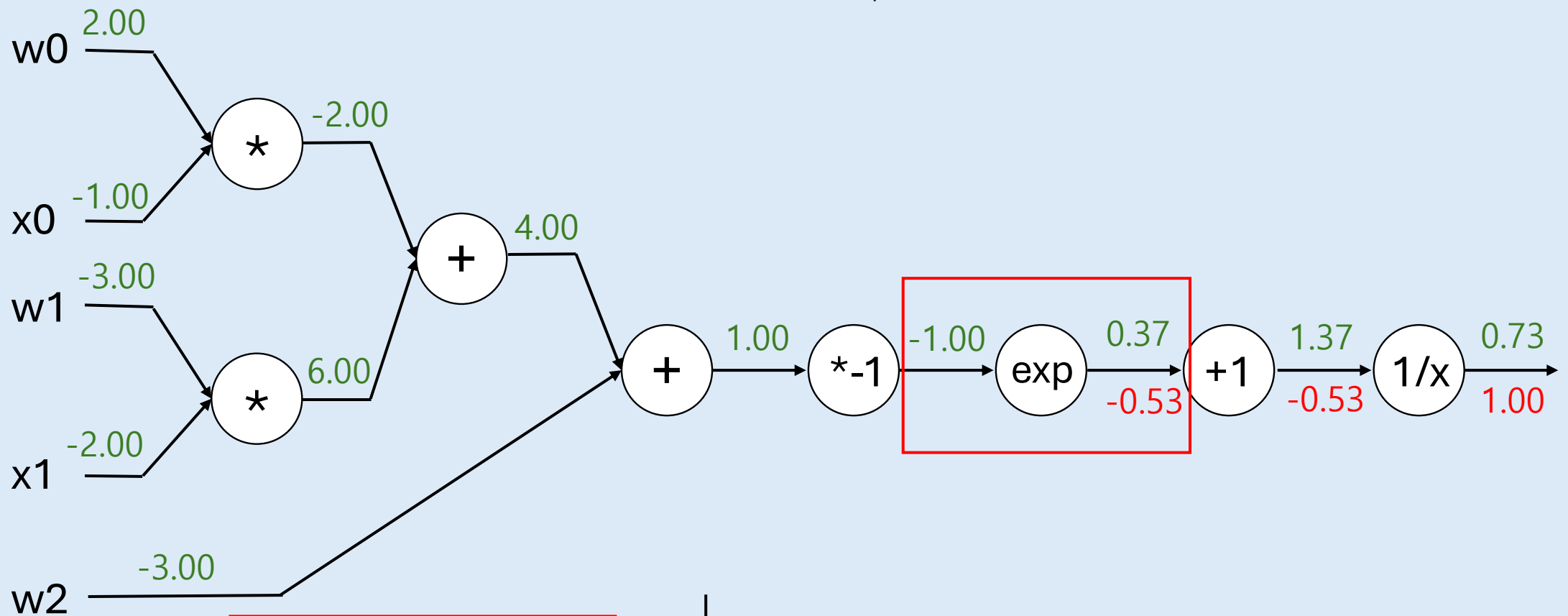
$$f(x) = e^x \longrightarrow \frac{df}{dx} = e^x$$

$$f_a(x) = ax \longrightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \longrightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \longrightarrow \frac{df}{dx} = 1$$

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



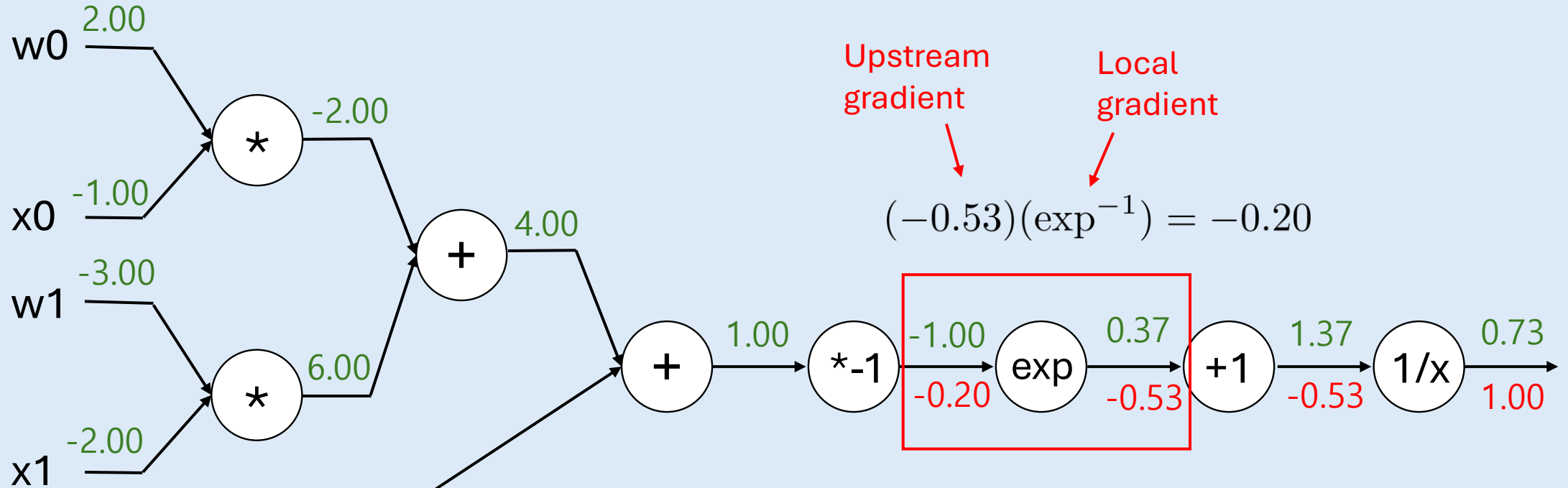
$$f(x) = e^x \longrightarrow \frac{df}{dx} = e^x$$

$$f_a(x) = ax \longrightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \longrightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \longrightarrow \frac{df}{dx} = 1$$

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



Upstream
gradient

Local
gradient

$$(-0.53)(\exp^{-1}) = -0.20$$

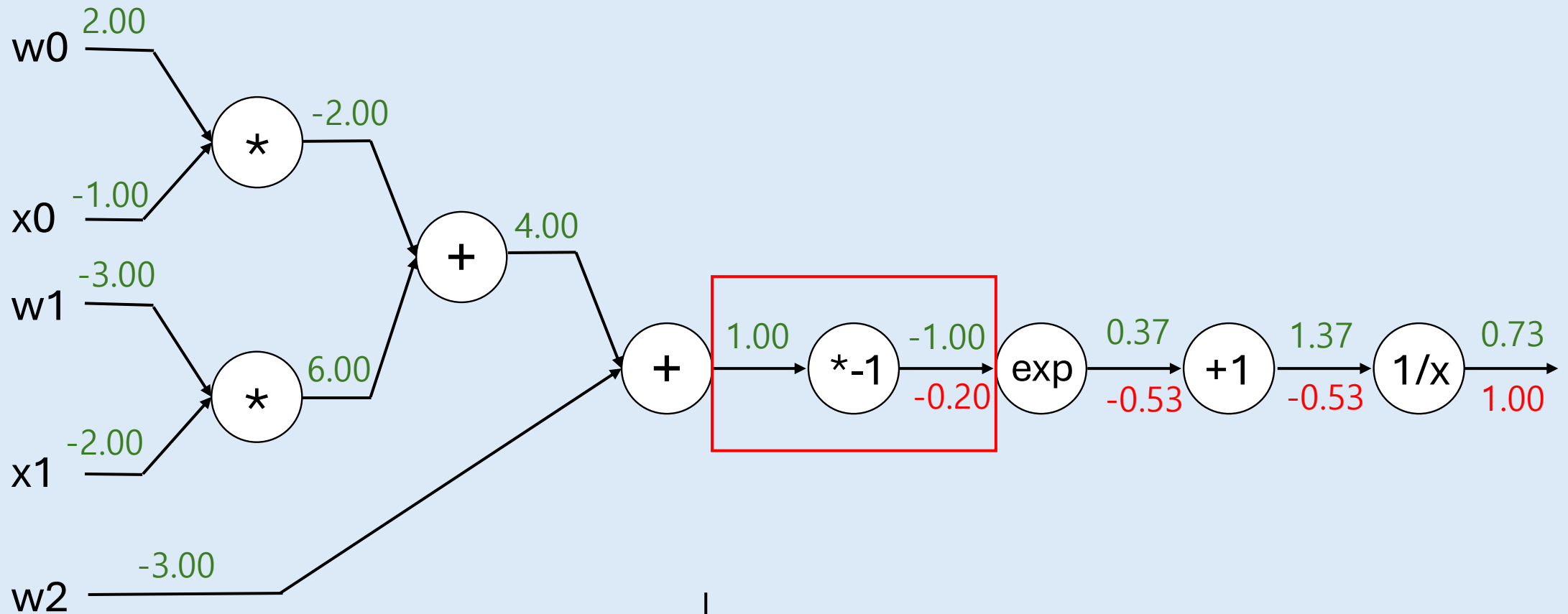
$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



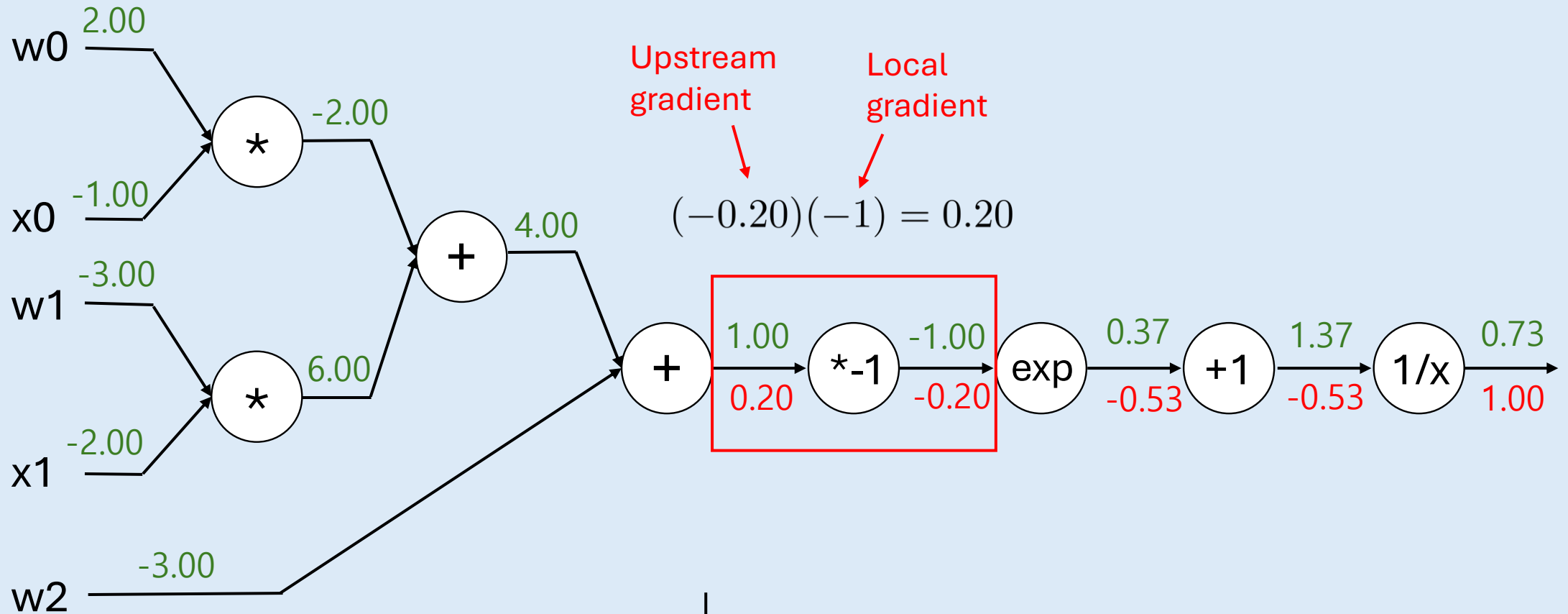
$$f(x) = e^x \longrightarrow \frac{df}{dx} = e^x$$

$$f_a(x) = ax \longrightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \longrightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \longrightarrow \frac{df}{dx} = 1$$

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



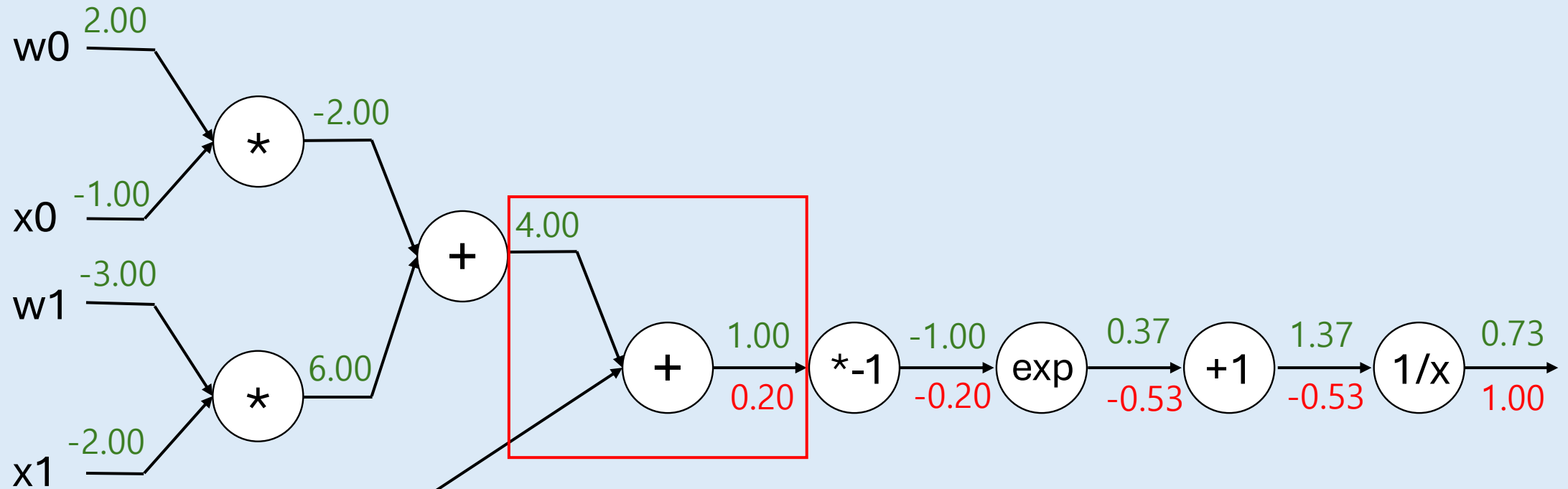
$$f(x) = e^x \longrightarrow \frac{df}{dx} = e^x$$

$$f_a(x) = ax \longrightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \longrightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \longrightarrow \frac{df}{dx} = 1$$

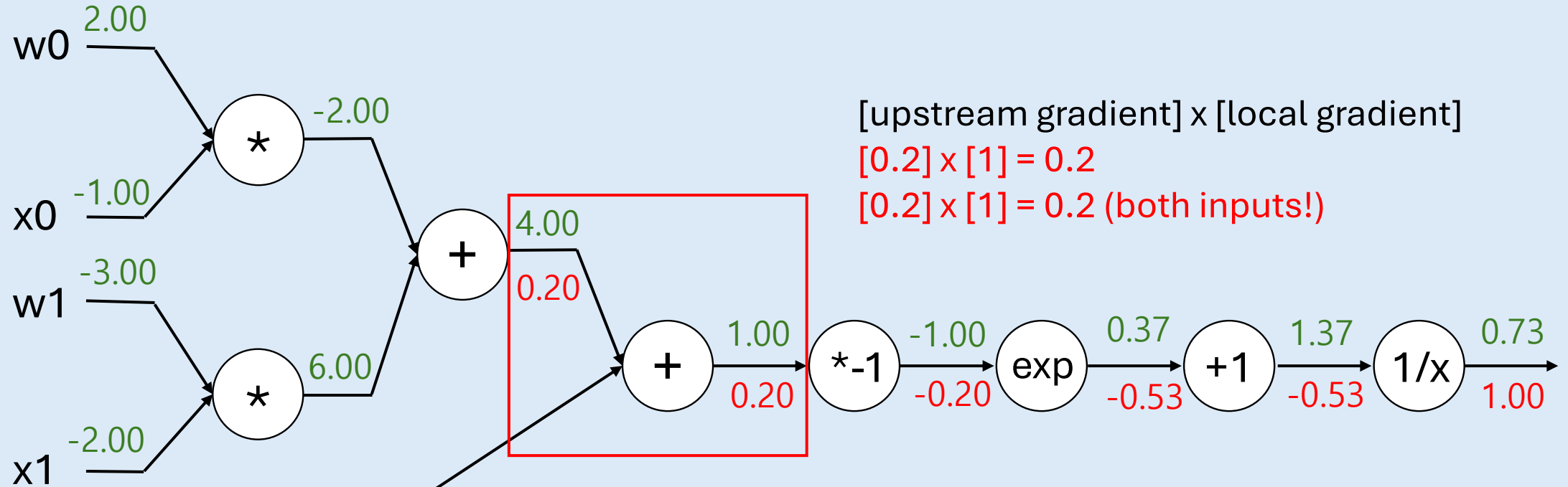
Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



$f(x) = e^x \longrightarrow \frac{df}{dx} = e^x$
 $f_a(x) = ax \longrightarrow \frac{df}{dx} = a$

$f(x) = \frac{1}{x} \longrightarrow \frac{df}{dx} = -1/x^2$
 $f_c(x) = c + x \longrightarrow \frac{df}{dx} = 1$

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



[upstream gradient] x [local gradient]

$[0.2] \times [1] = 0.2$

$[0.2] \times [1] = 0.2$ (both inputs!)

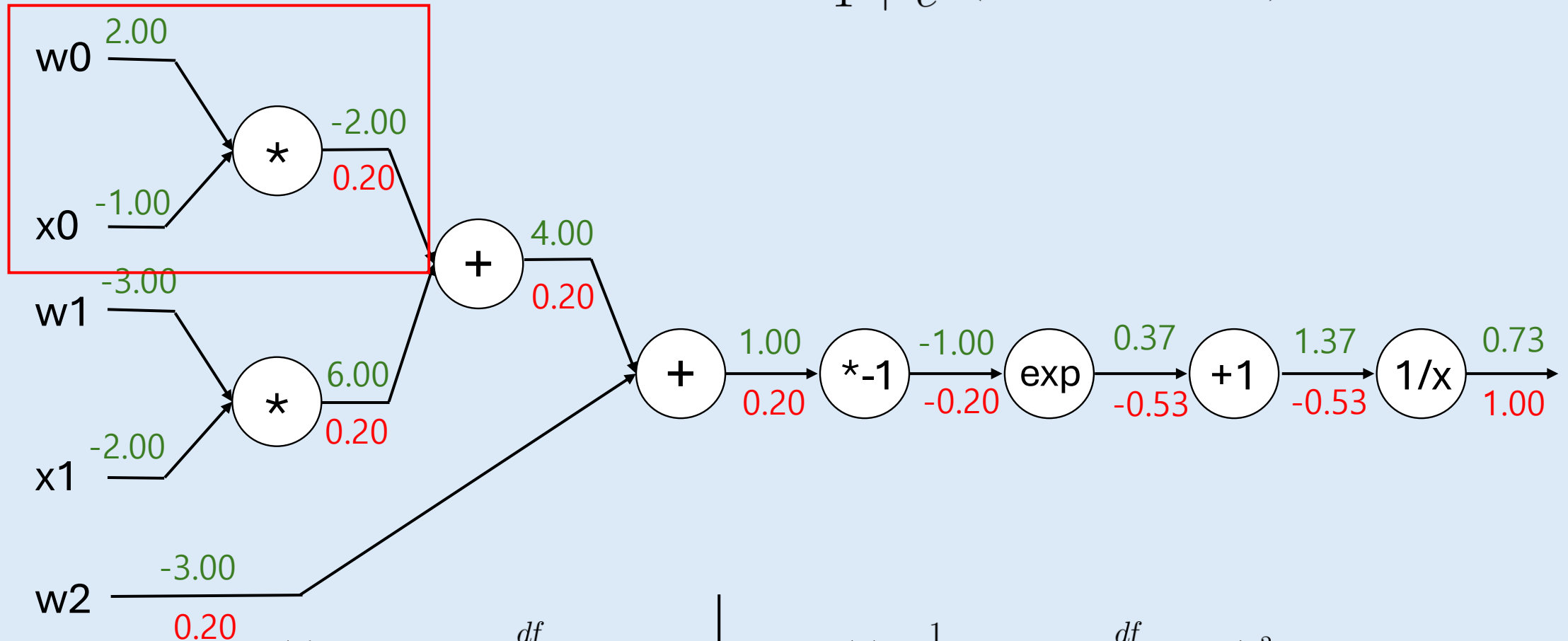
$$f(x) = e^x \longrightarrow \frac{df}{dx} = e^x$$

$$f_a(x) = ax \longrightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \longrightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \longrightarrow \frac{df}{dx} = 1$$

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



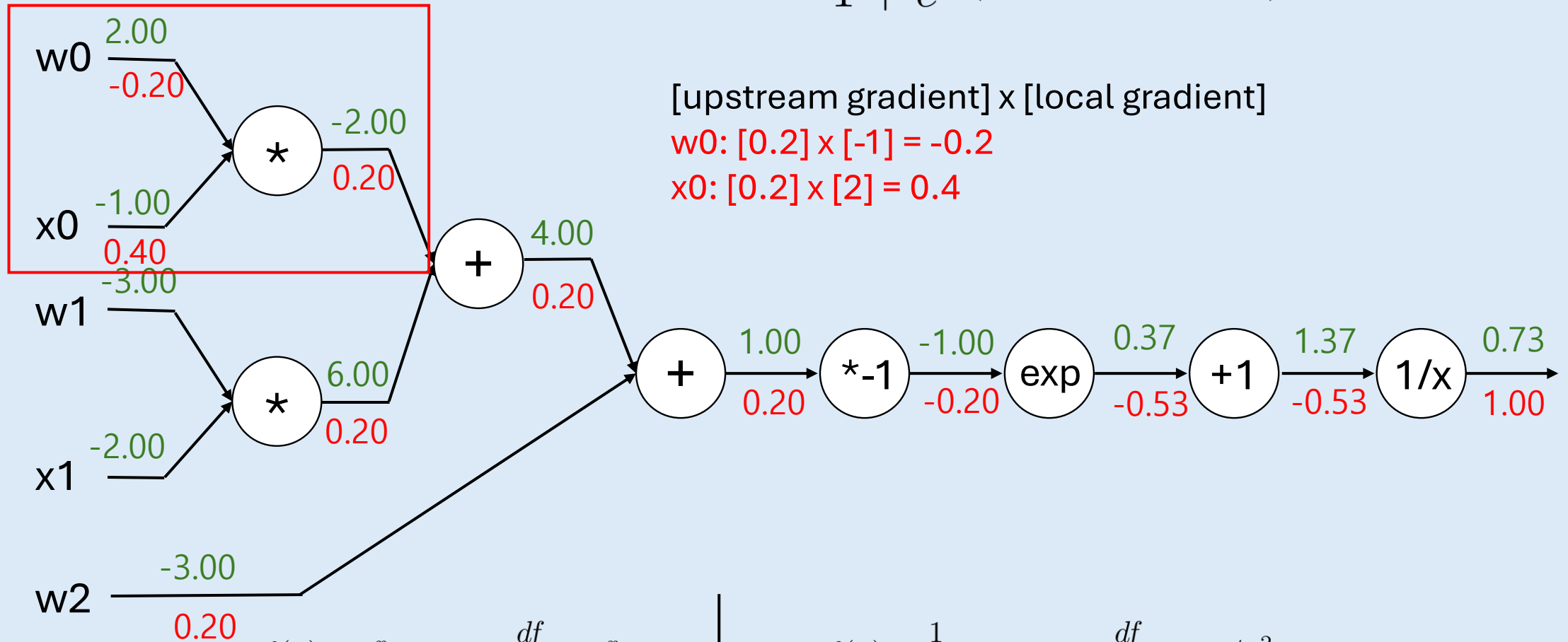
$$f(x) = e^x \longrightarrow \frac{df}{dx} = e^x$$

$$f_a(x) = ax \longrightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \longrightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \longrightarrow \frac{df}{dx} = 1$$

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



[upstream gradient] x [local gradient]

w_0 : $[0.2] \times [-1] = -0.2$

x_0 : $[0.2] \times [2] = 0.4$

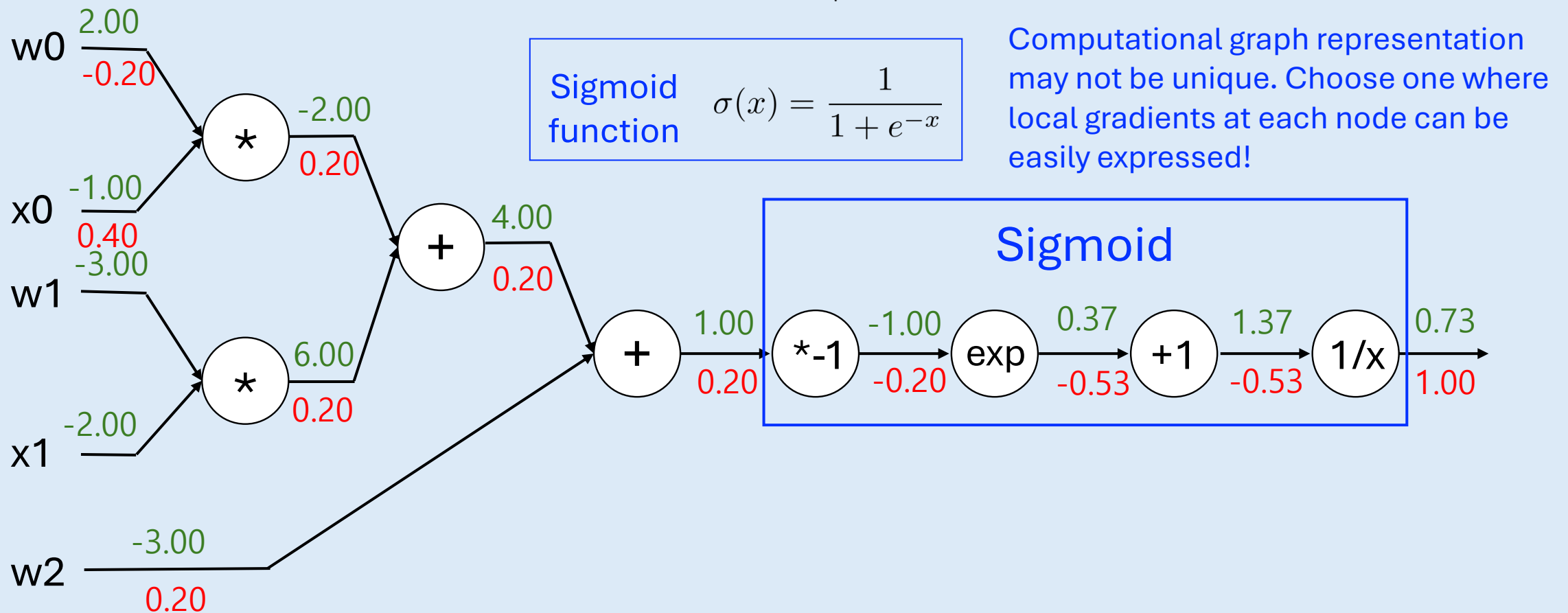
$$f(x) = e^x \longrightarrow \frac{df}{dx} = e^x$$

$$f_a(x) = ax \longrightarrow \frac{df}{dx} = a$$

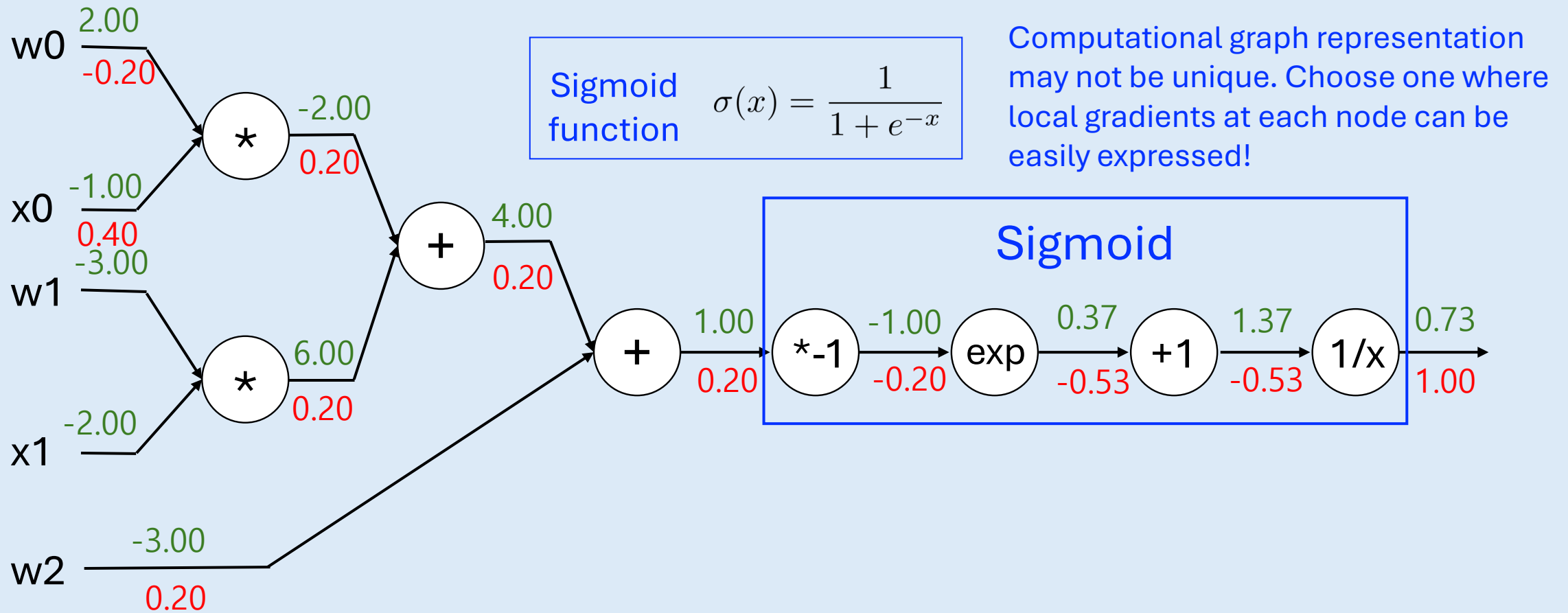
$$f(x) = \frac{1}{x} \longrightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \longrightarrow \frac{df}{dx} = 1$$

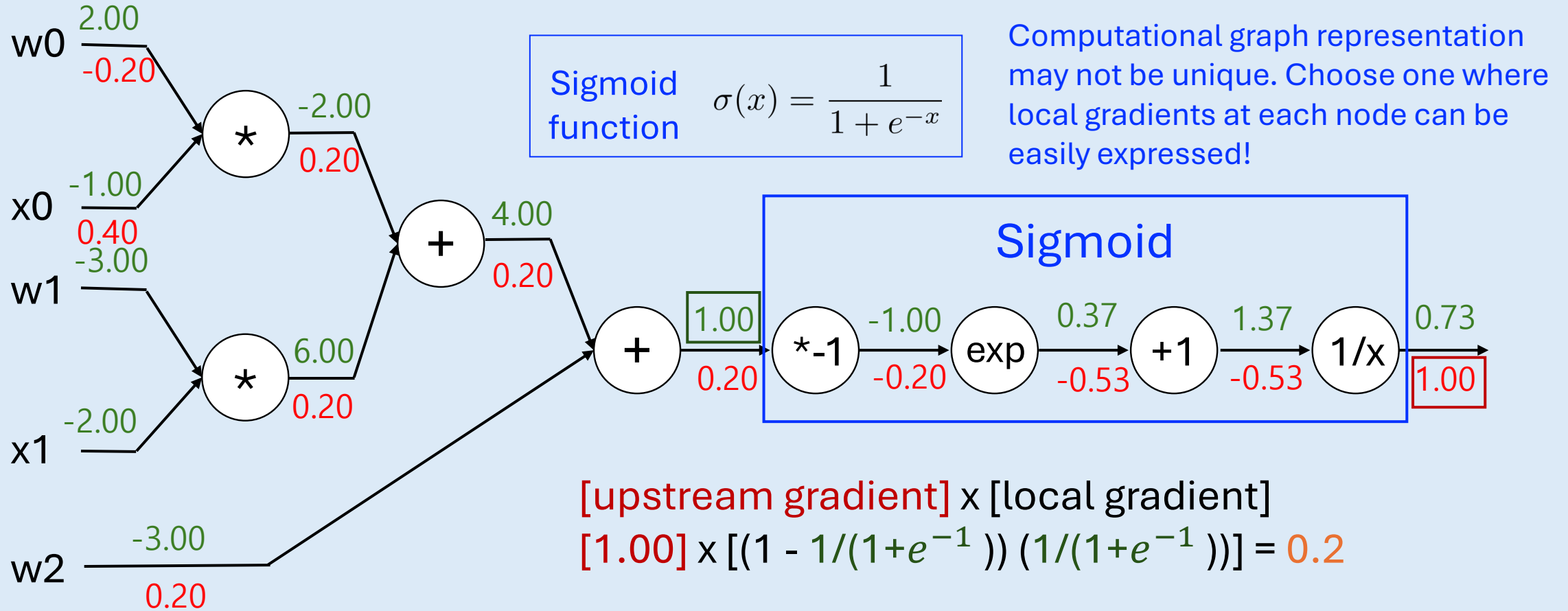
Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



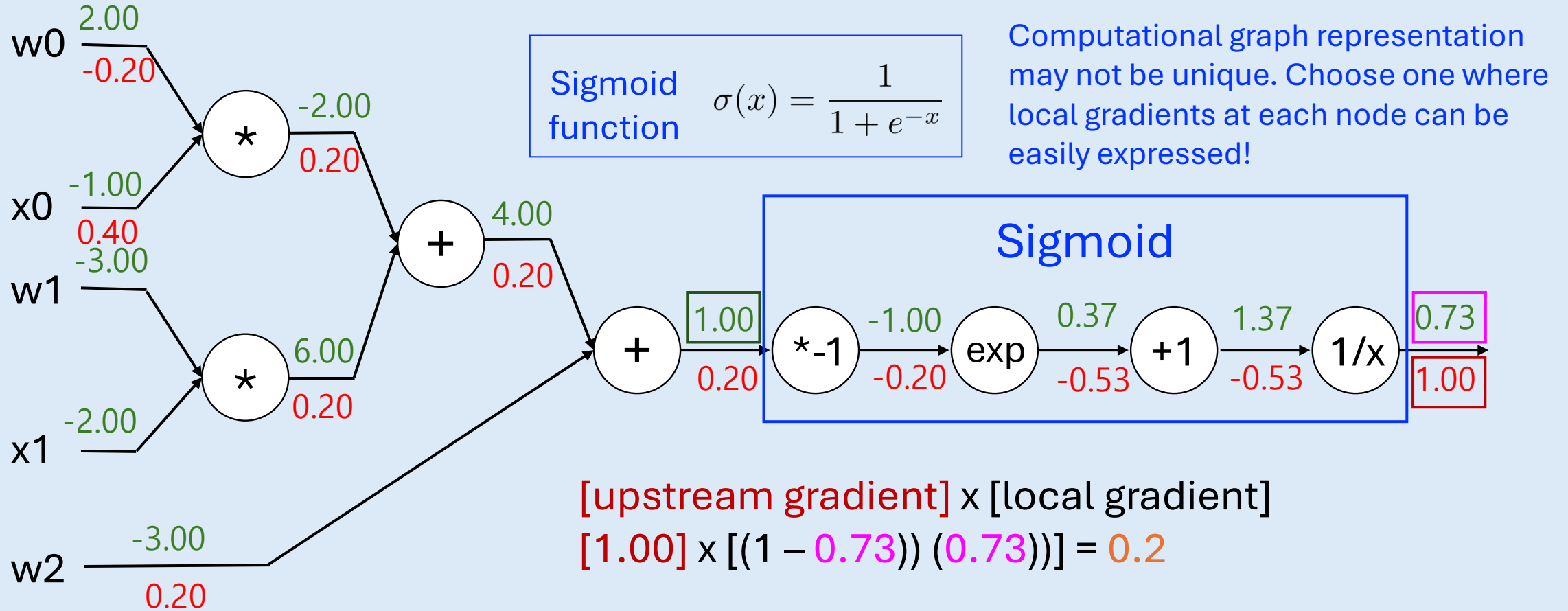
Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



Sigmoid
 local gradient:

$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x))\sigma(x)$$

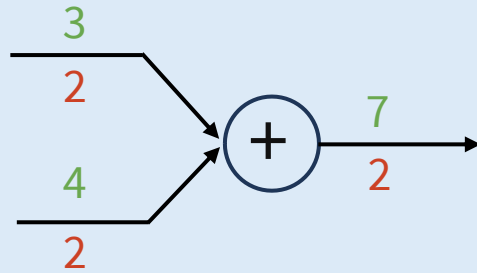
Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$



$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x))\sigma(x)$$

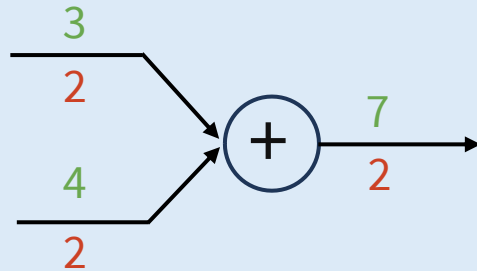
Patterns in gradient flow

add gate: gradient distributor

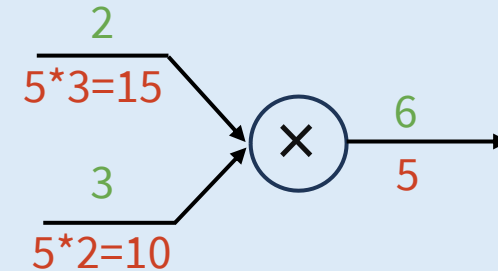


Patterns in gradient flow

add gate: gradient distributor

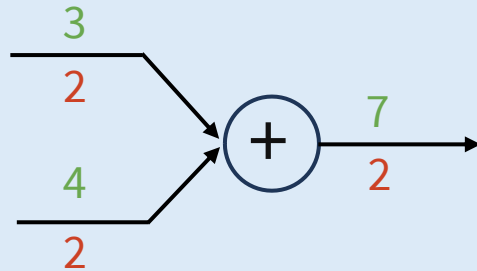


mul gate: “swap multiplier”

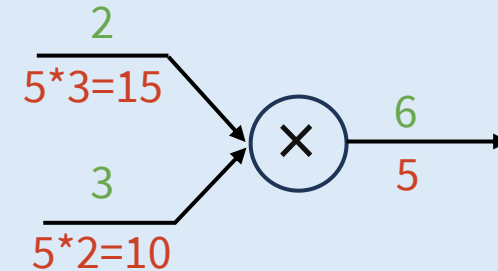


Patterns in gradient flow

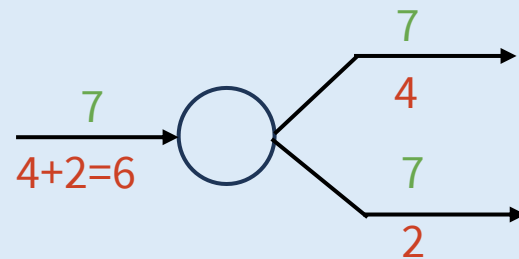
add gate: gradient distributor



mul gate: “swap multiplier”

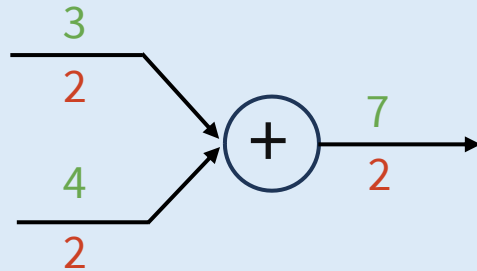


copy gate: gradient adder

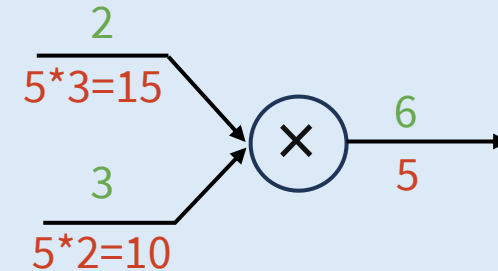


Patterns in gradient flow

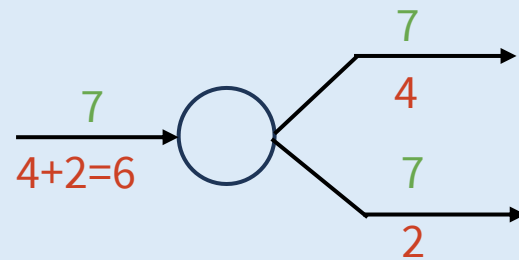
add gate: gradient distributor



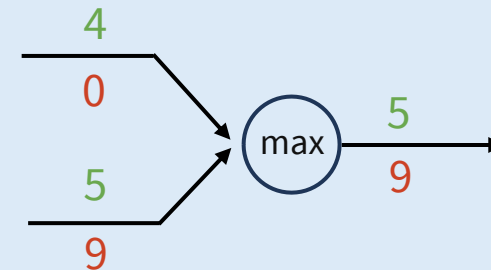
mul gate: “swap multiplier”



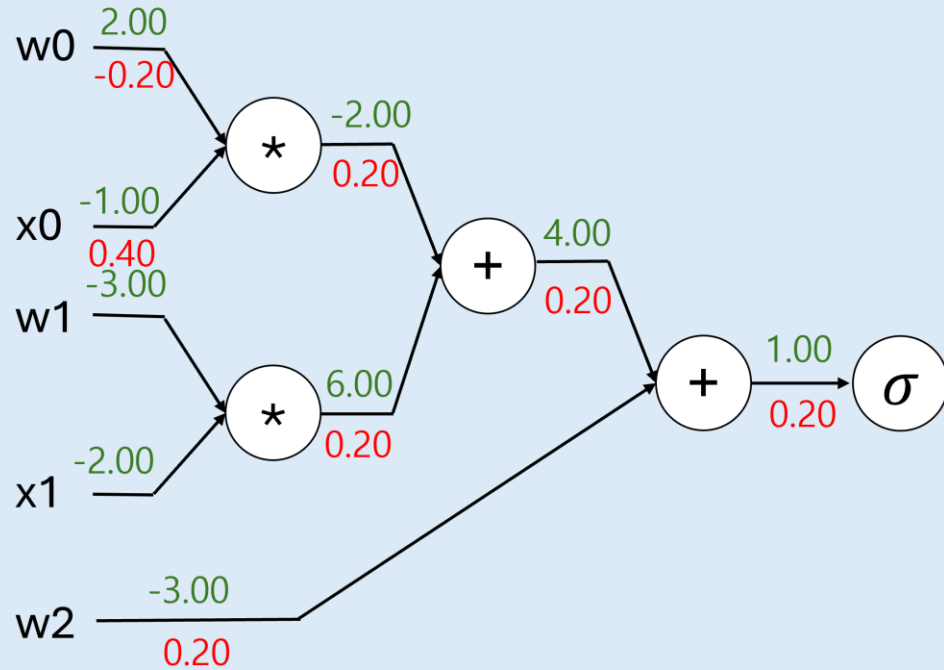
copy gate: gradient adder



max gate: gradient router



Backprop Implementation: “Flat” code



Forward pass:
Compute output

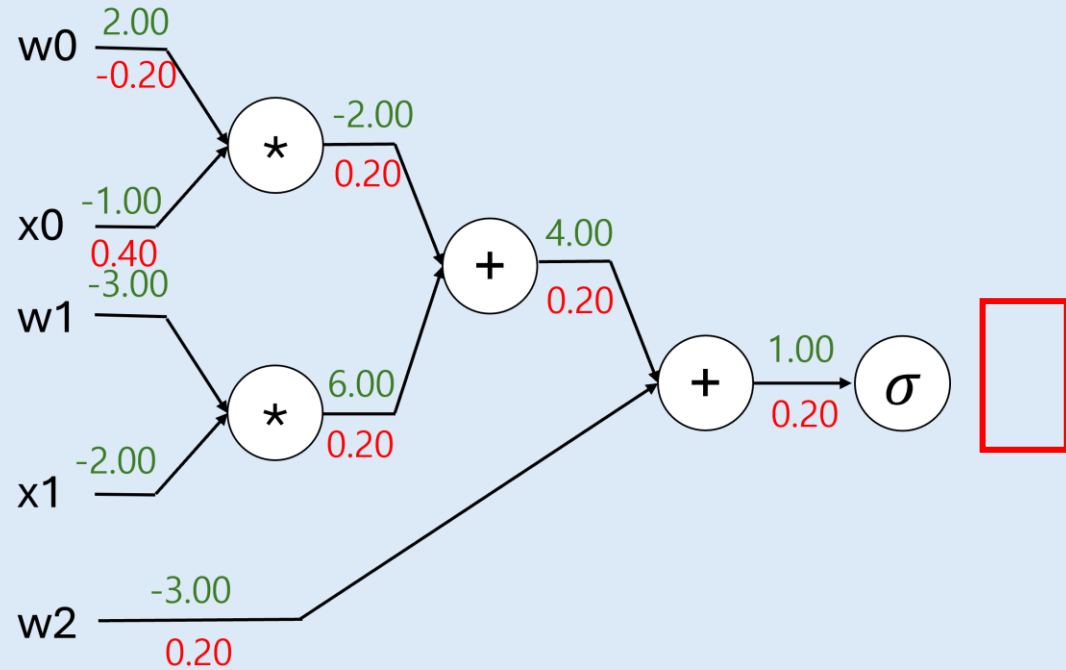
```
def f(w0, x0, w1, x1, w2):
```

```
    s0 = w0 * x0  
    s1 = w1 * x1  
    s2 = s0 + s1  
    s3 = s2 + w2  
    L = sigmoid(s3)
```

Backward pass:
Compute grads

```
    grad_L = 1.0  
    grad_s3 = grad_L * (1 - L) * L  
    grad_w2 = grad_s3  
    grad_s2 = grad_s3  
    grad_s0 = grad_s2  
    grad_s1 = grad_s2  
    grad_w1 = grad_s1 * x1  
    grad_x1 = grad_s1 * w1  
    grad_w0 = grad_s0 * x0  
    grad_x0 = grad_s0 * w0
```

Backprop Implementation: “Flat” code



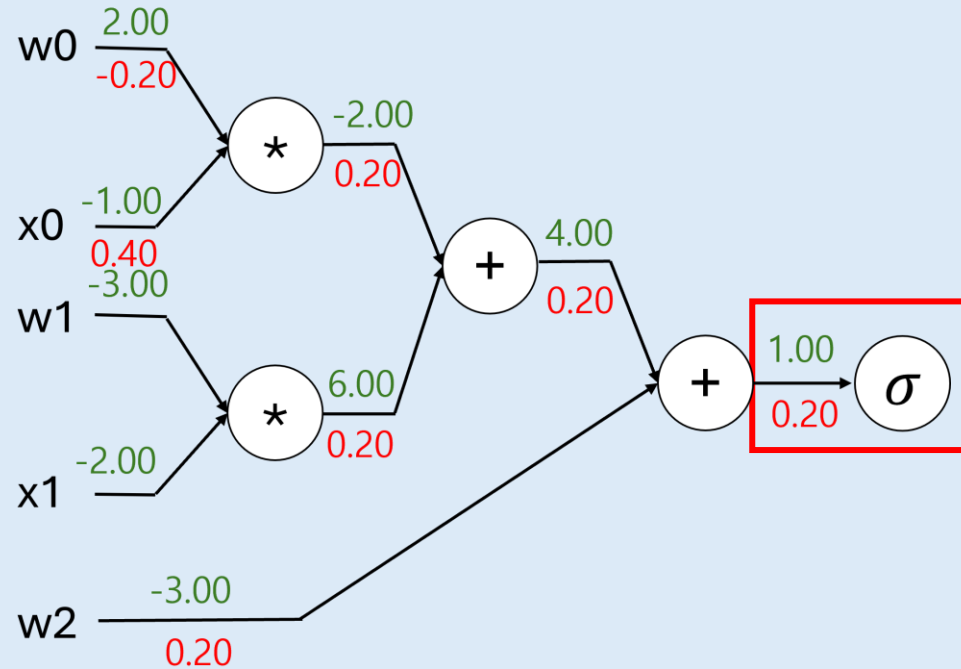
Forward pass:
Compute output

Base case

```
def f(w0, x0, w1, x1, w2):
    s0 = w0 * x0
    s1 = w1 * x1
    s2 = s0 + s1
    s3 = s2 + w2
    L = sigmoid(s3)

    grad_L = 1.0
    grad_s3 = grad_L * (1 - L) * L
    grad_w2 = grad_s3
    grad_s2 = grad_s3
    grad_s0 = grad_s2
    grad_s1 = grad_s2
    grad_w1 = grad_s1 * x1
    grad_x1 = grad_s1 * w1
    grad_w0 = grad_s0 * x0
    grad_x0 = grad_s0 * w0
```

Backprop Implementation: “Flat” code



Forward pass:
Compute output

Sigmoid

```
def f(w0, x0, w1, x1, w2):
```

```
    s0 = w0 * x0
```

```
    s1 = w1 * x1
```

```
    s2 = s0 + s1
```

```
    s3 = s2 + w2
```

```
    L = sigmoid(s3)
```

```
    grad_L = 1.0
```

```
    grad_s3 = grad_L * (1 - L) * L
```

```
    grad_w2 = grad_s3
```

```
    grad_s2 = grad_s3
```

```
    grad_s0 = grad_s2
```

```
    grad_s1 = grad_s2
```

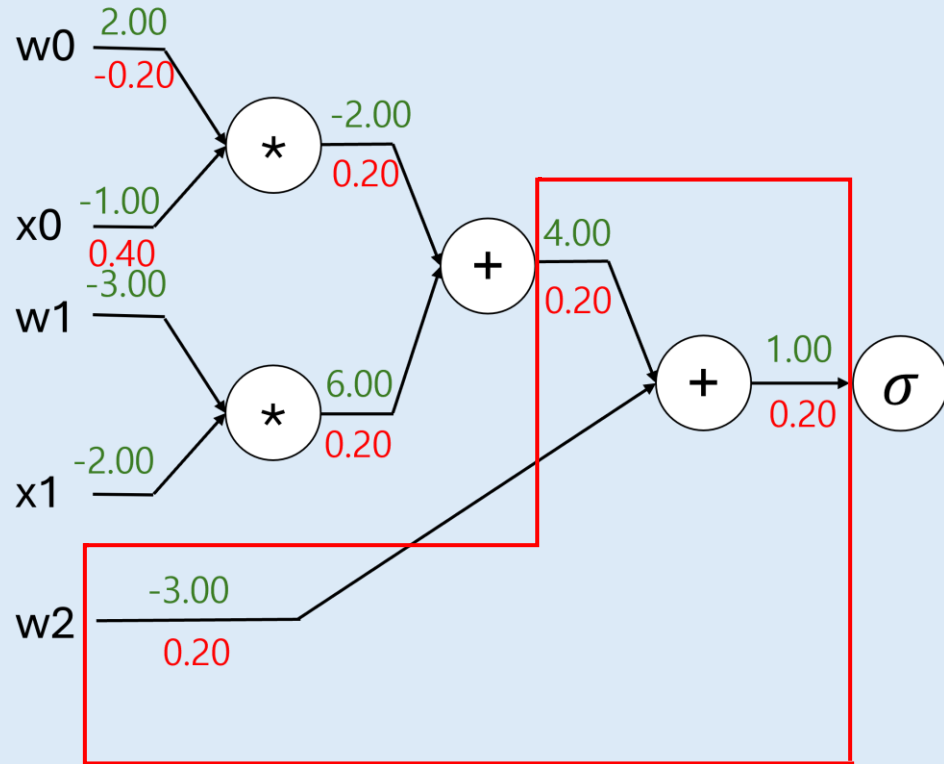
```
    grad_w1 = grad_s1 * x1
```

```
    grad_x1 = grad_s1 * w1
```

```
    grad_w0 = grad_s0 * x0
```

```
    grad_x0 = grad_s0 * w0
```

Backprop Implementation: “Flat” code



Forward pass:
Compute output

```
def f(w0, x0, w1, x1, w2):
```

```
    s0 = w0 * x0
```

```
    s1 = w1 * x1
```

```
    s2 = s0 + s1
```

```
    s3 = s2 + w2
```

```
    L = sigmoid(s3)
```

```
    grad_L = 1.0
```

```
    grad_s3 = grad_L * (1 - L) * L
```

```
    grad_w2 = grad_s3
```

```
    grad_s2 = grad_s3
```

```
    grad_s0 = grad_s2
```

```
    grad_s1 = grad_s2
```

```
    grad_w1 = grad_s1 * x1
```

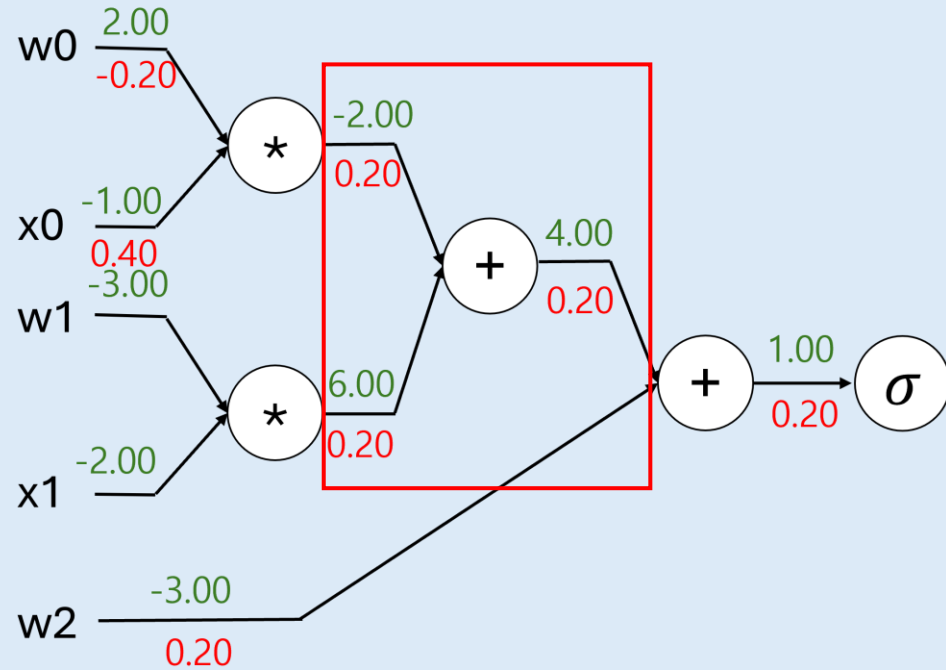
```
    grad_x1 = grad_s1 * w1
```

```
    grad_w0 = grad_s0 * x0
```

```
    grad_x0 = grad_s0 * w0
```

Add gate

Backprop Implementation: “Flat” code



Forward pass:
Compute output

```
def f(w0, x0, w1, x1, w2):
```

```
    s0 = w0 * x0
```

```
    s1 = w1 * x1
```

```
    s2 = s0 + s1
```

```
    s3 = s2 + w2
```

```
    L = sigmoid(s3)
```

```
    grad_L = 1.0
```

```
    grad_s3 = grad_L * (1 - L) * L
```

```
    grad_w2 = grad_s3
```

```
    grad_s2 = grad_s3
```

```
    grad_s0 = grad_s2
```

```
    grad_s1 = grad_s2
```

```
    grad_w1 = grad_s1 * x1
```

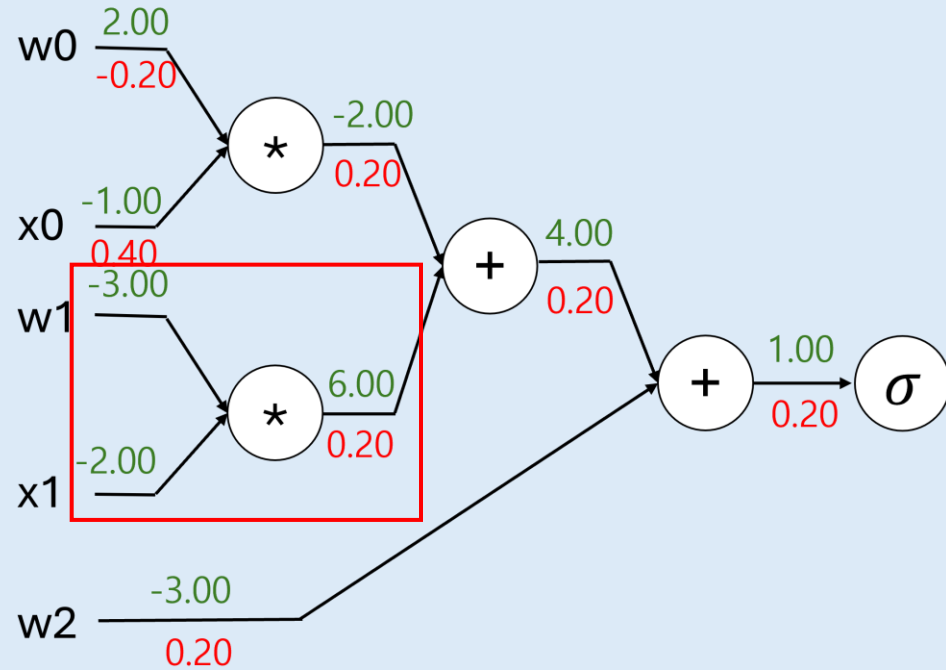
```
    grad_x1 = grad_s1 * w1
```

```
    grad_w0 = grad_s0 * x0
```

```
    grad_x0 = grad_s0 * w0
```

Add gate

Backprop Implementation: “Flat” code



Forward pass:
Compute output

```
def f(w0, x0, w1, x1, w2):
```

```
    s0 = w0 * x0
```

```
    s1 = w1 * x1
```

```
    s2 = s0 + s1
```

```
    s3 = s2 + w2
```

```
    L = sigmoid(s3)
```

```
    grad_L = 1.0
```

```
    grad_s3 = grad_L * (1 - L) * L
```

```
    grad_w2 = grad_s3
```

```
    grad_s2 = grad_s3
```

```
    grad_s0 = grad_s2
```

```
    grad_s1 = grad_s2
```

```
    grad_w1 = grad_s1 * x1
```

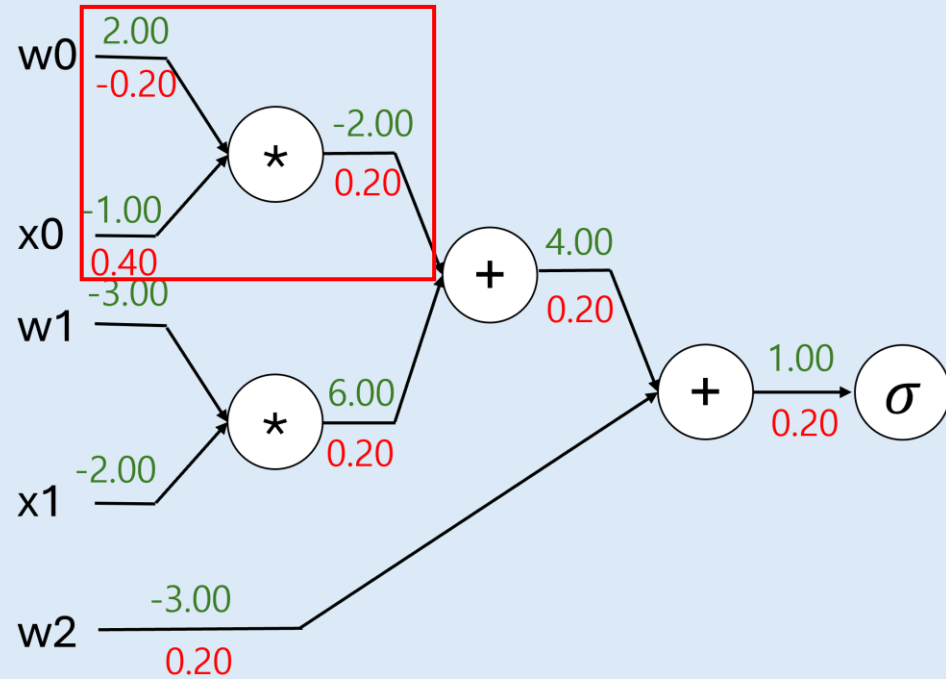
```
    grad_x1 = grad_s1 * w1
```

```
    grad_w0 = grad_s0 * x0
```

```
    grad_x0 = grad_s0 * w0
```

Add gate

Backprop Implementation: “Flat” code



Forward pass:
Compute output

```
def f(w0, x0, w1, x1, w2):
```

```
    s0 = w0 * x0
```

```
    s1 = w1 * x1
```

```
    s2 = s0 + s1
```

```
    s3 = s2 + w2
```

```
    L = sigmoid(s3)
```

```
    grad_L = 1.0
```

```
    grad_s3 = grad_L * (1 - L) * L
```

```
    grad_w2 = grad_s3
```

```
    grad_s2 = grad_s3
```

```
    grad_s0 = grad_s2
```

```
    grad_s1 = grad_s2
```

```
    grad_w1 = grad_s1 * x1
```

```
    grad_x1 = grad_s1 * w1
```

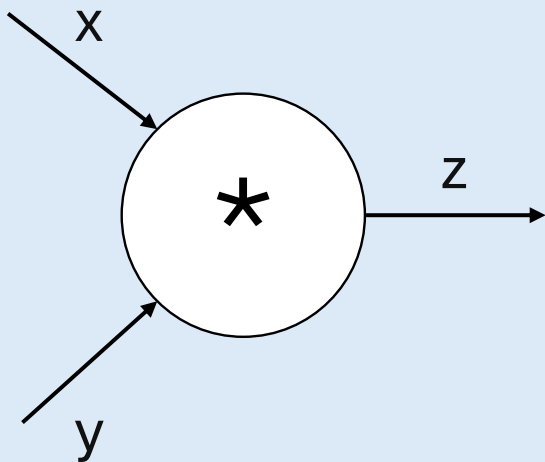
```
    grad_w0 = grad_s0 * x0
```

```
    grad_x0 = grad_s0 * w0
```

Add gate

Modularized implementation: forward / backward API

Gate / Node / Function object: Actual PyTorch code



(x,y,z are scalars)

```
class Multiply(torch.autograd.Function):  
    @staticmethod  
    def forward(ctx, x, y):  
        ctx.save_for_backward(x, y)  ←  
        z = x * y  
        return z  
  
    @staticmethod  
    def backward(ctx, grad_z):  ←  
        x, y = ctx.saved_tensors  
        grad_x = y * grad_z  # dz/dx * dL/dz  
        grad_y = x * grad_z  # dz/dy * dL/dz  
        return grad_x, grad_y
```

Need to cache
some values for
use in backward

Upstream
gradient

Multiply upstream
and local gradients

Example: PyTorch operators

pytorch / aten / src / THNN / generic /

⚠ This commit does not belong to any branch on this repository, and may belong to a fork outside of the repository.

ssnl cherry pick #9500 and #9590 into 0.4.1 (#9599) 3684cc4 · 8 years ago History

Name	Last commit message	Last commit date
..		
Abs.c	move TH* folders into aten/src	9 years ago
AbsCriterion.c	combine size_average and reduce args in loss fun...	8 years ago
BCECriterion.c	combine size_average and reduce args in loss fun...	8 years ago
BatchNormalization.c	[ready] Layer Normalization (#4922)	8 years ago
ClassNLLCriterion.c	combine size_average and reduce args in loss fun...	8 years ago
Col2Im.c	cherry pick #9500 and #9590 into 0.4.1 (#9599)	8 years ago
DistKLDivCriterion.c	combine size_average and reduce args in loss fun...	8 years ago
ELU.c	Move SELU to ATen (#4269)	9 years ago
FeatureLPPooling.c	Compatibility: write nDimension/_nDimension co...	8 years ago
FusedRNKernel.c	move TH* folders into aten/src	9 years ago
GatedLinearUnit.c	Port all indirect calls of resizeNdLegacy to resize...	8 years ago
HardShrink.c	propagate nan in some activations (#8033)	8 years ago
HardTanh.c	Prepare for moving 0-sized dimensions in TH/TH...	8 years ago
Im2Col.c	cherry pick #9500 and #9590 into 0.4.1 (#9599)	8 years ago
IndexLinear.c	Compatibility: write nDimension/_nDimension co...	8 years ago
L1Cost.c	move TH* folders into aten/src	9 years ago
LeakyReLU.c	Optimize LeakyReLU and PReLU 'forward' functio...	8 years ago

...

[link](#)

Sigmoid.c	move TH* folders into aten/src	9 years ago
SmoothL1Criterion.c	combine size_average and reduce args in loss fun...	8 years ago
SoftMarginCriterion.c	combine size_average and reduce args in loss fun...	8 years ago
SoftPlus.c	move TH* folders into aten/src	9 years ago
SoftShrink.c	move TH* folders into aten/src	9 years ago
SparseLinear.c	Support n-dimensional empty tensors in (most of...	8 years ago
SpatialAdaptiveAveragePooling.c	Support n-dimensional empty tensors in (most of...	8 years ago
SpatialAdaptiveMaxPooling.c	Support n-dimensional empty tensors in (most of...	8 years ago
SpatialAveragePooling.c	Support n-dimensional empty tensors in (most of...	8 years ago
SpatialClassNLLCriterion.c	combine size_average and reduce args in loss fun...	8 years ago
SpatialConvolutionLocal.c	Support n-dimensional empty tensors in (most of...	8 years ago
SpatialConvolutionMM.c	Support n-dimensional empty tensors in (most of...	8 years ago
SpatialConvolutionMap.c	Support n-dimensional empty tensors in (most of...	8 years ago
SpatialDilatedConvolution.c	cherry pick #9500 and #9590 into 0.4.1 (#9599)	8 years ago
SpatialDilatedMaxPooling.c	Support n-dimensional empty tensors in (most of...	8 years ago
SpatialFractionalMaxPooling.c	Support n-dimensional empty tensors in (most of...	8 years ago
SpatialFullConvolution.c	move TH* folders into aten/src	9 years ago
SpatialFullConvolutionMap.c	Support n-dimensional empty tensors in (most of...	8 years ago
SpatialFullDilatedConvolution.c	cherry pick #9500 and #9590 into 0.4.1 (#9599)	8 years ago
SpatialGridSamplerBilinear.c	Support n-dimensional empty tensors in (most of...	8 years ago



zdevito and ezyang move TH* folders into aten/src

Code

Blame

27 lines (24 loc) · 642 Bytes

```
1  #ifndef TH_GENERIC_FILE
2  #define TH_GENERIC_FILE "generic/Sigmoid.c"
3  #else
4
5  void THNN_(Sigmoid_updateOutput)(
6      THNNState *state,
7      THTensor *input,
8      THTensor *output)
9  {
10     THTensor_(sigmoid)(output, input);
11 }
12
13 void THNN_(Sigmoid_updateGradInput)(
14     THNNState *state,
15     THTensor *gradOutput,
16     THTensor *gradInput,
17     THTensor *output)
18 {
19     THNN_CHECK_NELEMENT(output, gradOutput);
20     THTensor_(resizeAs)(gradInput, output);
21     TH_TENSOR_APPLY3(real, gradInput, real, gradOutput, real, output,
22         real z = *output_data;
23         *gradInput_data = *gradOutput_data * (1. - z) * z;
24     );
25 }
26
27 #endif
```

Forward

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$





zdevito and ezyang move TH* folders into aten/src

Code Blame 27 lines (24 loc) · 642 Bytes

```

1  #ifndef TH_GENERIC_FILE
2  #define TH_GENERIC_FILE "generic/Sigmoid.c"
3  #else
4
5  void THNN_(Sigmoid_updateOutput)(
6      THNNState *state,
7      THTensor *input,
8      THTensor *output)
9  {
10     THTensor_(sigmoid)(output, input);
11 }
12
13 void THNN_(Sigmoid_updateGradInput)(
14     THNNState *state,
15     THTensor *gradOutput,
16     THTensor *gradInput,
17     THTensor *output)
18 {
19     THNN_CHECK_NELEMENT(output, gradOutput);
20     THTensor_(resizeAs)(gradInput, output);
21     TH_TENSOR_APPLY3(real, gradInput, real, gradOutput, real, output,
22         real z = *output_data;
23         *gradInput_data = *gradOutput_data * (1. - z) * z;
24     );
25 }
26
27 #endif
    
```

Forward

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



PyTorch sigmoid layer

```

static void sigmoid_kernel(TensorIterator& iter) {
    AT_DISPATCH_FLOATING_TYPES(iter.dtype(), "sigmoid_cpu", [&]() {
        unary_kernel_vec(
            iter,
            [=](scalar_t a) -> scalar_t { return (1 / (1 + std::exp((-a)))); },
            [=](Vec256<scalar_t> a) {
                a = Vec256<scalar_t>((scalar_t)(0)) - a;
                a = a.exp();
                a = Vec256<scalar_t>((scalar_t)(1)) + a;
                a = a.reciprocal();
                return a;
            });
    });
}
    
```

Forward
actually
defined
elsewhere...



Code

Blame

27 lines (24 loc) · 642 Bytes

```

1  #ifndef TH_GENERIC_FILE
2  #define TH_GENERIC_FILE "generic/Sigmoid.c"
3  #else

```

```

5  void THNN_(Sigmoid_updateOutput)(
6      THNNState *state,
7      THTensor *input,
8      THTensor *output)
9  {
10     THTensor_(sigmoid)(output, input);
11 }

```

Forward

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

```

13 void THNN_(Sigmoid_updateGradInput)(
14     THNNState *state,
15     THTensor *gradOutput,
16     THTensor *gradInput,
17     THTensor *output)
18 {
19     THNN_CHECK_NELEMENT(output, gradOutput);
20     THTensor_(resizeAs)(gradInput, output);
21     TH_TENSOR_APPLY3(real, gradInput, real, gradOutput, real, output,
22         real z = *output_data;
23         *gradInput_data = *gradOutput_data * (1. - z) * z;
24 );
25 }
26
27 #endif

```

Backward

$$(1 - \sigma(x))\sigma(x)$$

PyTorch sigmoid layer

```

static void sigmoid_kernel(TensorIterator& iter) {
    AT_DISPATCH_FLOATING_TYPES(iter.dtype(), "sigmoid_cpu", [&]() {
        unary_kernel_vec(
            iter,
            [=](scalar_t a) -> scalar_t { return (1 / (1 + std::exp((-a)))); },
            [=](Vec256<scalar_t> a) {
                a = Vec256<scalar_t>((scalar_t)(0)) - a;
                a = a.exp();
                a = Vec256<scalar_t>((scalar_t)(1)) + a;
                a = a.reciprocal();
                return a;
            });
    });
}

```

Forward
actually
defined
elsewhere...

So far: backprop with scalars

What about vector-valued functions?



Recap: notation for vector calculus

XXX In this class we will use column convention...

Row convention is also possible but...

Different form of chain rule, but same formula

It may confuses. But remember that dimension

In this lecture we will use round z round $x = [\text{some matrix}]...$



Recap: vector derivatives

Scalar to Scalar

Vector to Scalar

Vector to Scalar

$$x \in \mathbb{R}, y \in \mathbb{R}$$

Regular derivative:

$$\frac{\partial y}{\partial x} \in \mathbb{R}$$

If $\mathbf{x}$ changes by a small amount, how much will $\mathbf{y}$ change?

Recap: vector derivatives

Scalar to Scalar

$$x \in \mathbb{R}, y \in \mathbb{R}$$

Regular derivative:

$$\frac{\partial y}{\partial x} \in \mathbb{R}$$

If $\mathbf{x}$ changes by a small amount, how much will $\mathbf{y}$ change?

Vector to Scalar

$$x \in \mathbb{R}^N, y \in \mathbb{R}$$

Derivative is Gradient:

$$\frac{\partial y}{\partial x} \in \mathbb{R}^N \quad \left(\frac{\partial y}{\partial x} \right)_n = \frac{\partial y}{\partial x_n}$$

For each element of $\mathbf{x}$, if it changes by a small amount then how much will $\mathbf{y}$ change?

Vector to Scalar

Recap: vector derivatives

Scalar to Scalar

$$x \in \mathbb{R}, y \in \mathbb{R}$$

Regular derivative:

$$\frac{\partial y}{\partial x} \in \mathbb{R}$$

If $\mathbf{x}$ changes by a small amount, how much will $\mathbf{y}$ change?

Vector to Scalar

$$x \in \mathbb{R}^N, y \in \mathbb{R}$$

Derivative is Gradient:

$$\frac{\partial y}{\partial x} \in \mathbb{R}^N \quad \left(\frac{\partial y}{\partial x} \right)_n = \frac{\partial y}{\partial x_n}$$

For each element of $\mathbf{x}$, if it changes by a small amount then how much will $\mathbf{y}$ change?

Vector to Vector

$$x \in \mathbb{R}^N, y \in \mathbb{R}^M$$

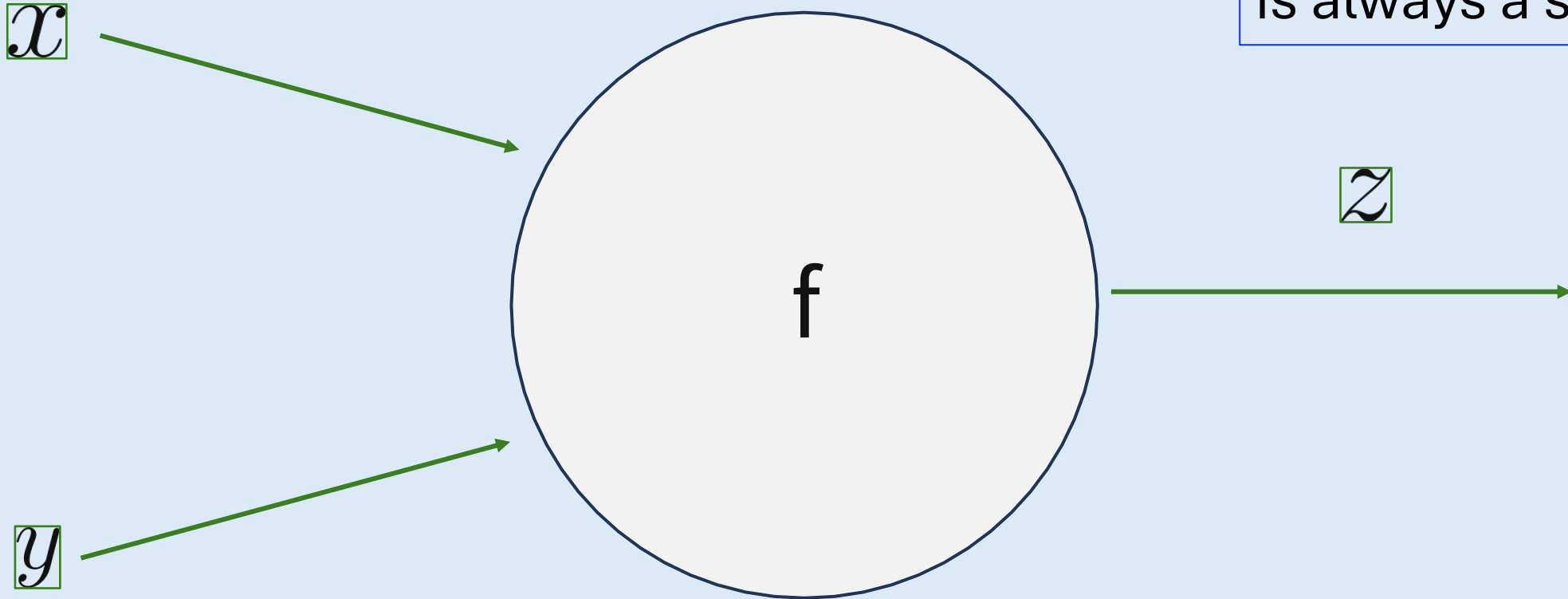
Derivative is Jacobian:

$$\frac{\partial y}{\partial x} \in \mathbb{R}^{N \times M} \quad \left(\frac{\partial y}{\partial x} \right)_{n,m} = \frac{\partial y_m}{\partial x_n}$$

For each element of $\mathbf{x}$, if it changes by a small amount then how much will each element of $\mathbf{y}$ change?

Backprop with vectors

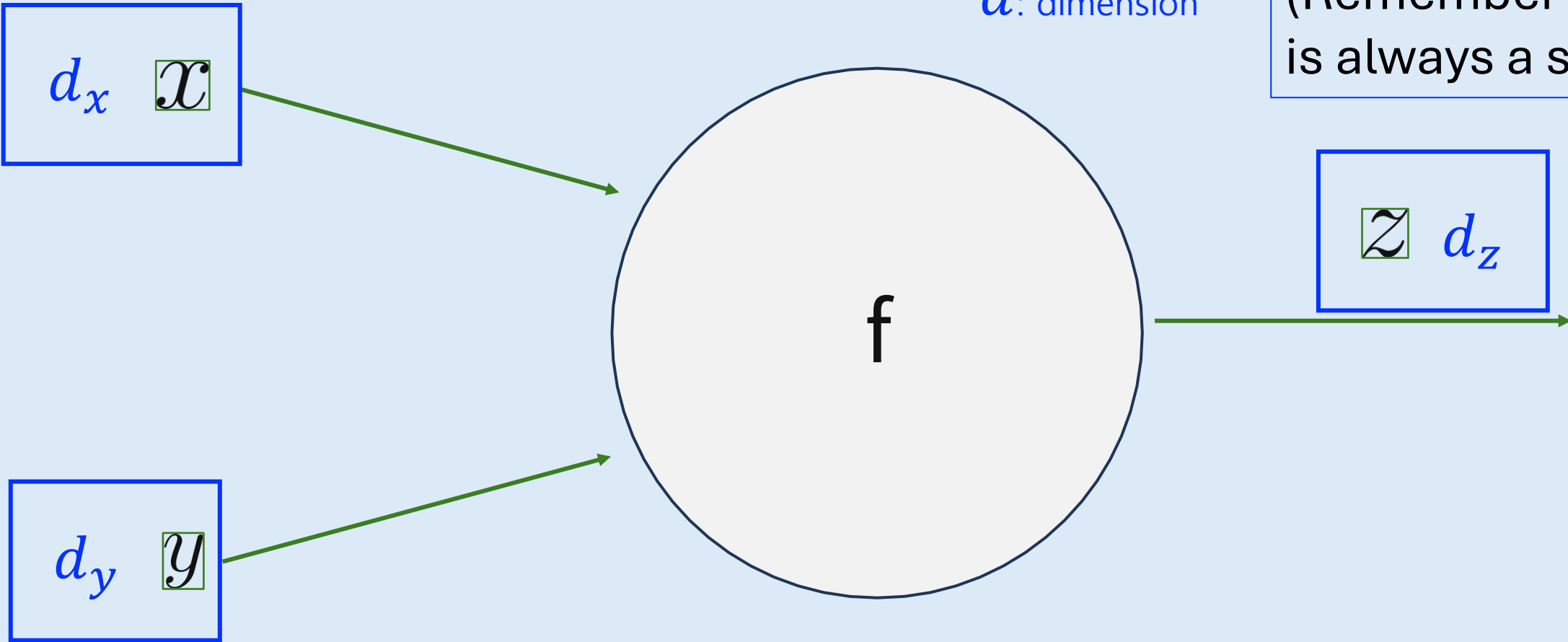
Loss L still a scalar!
(Remember that loss
is always a scalar)



Backprop with vectors

d : dimension

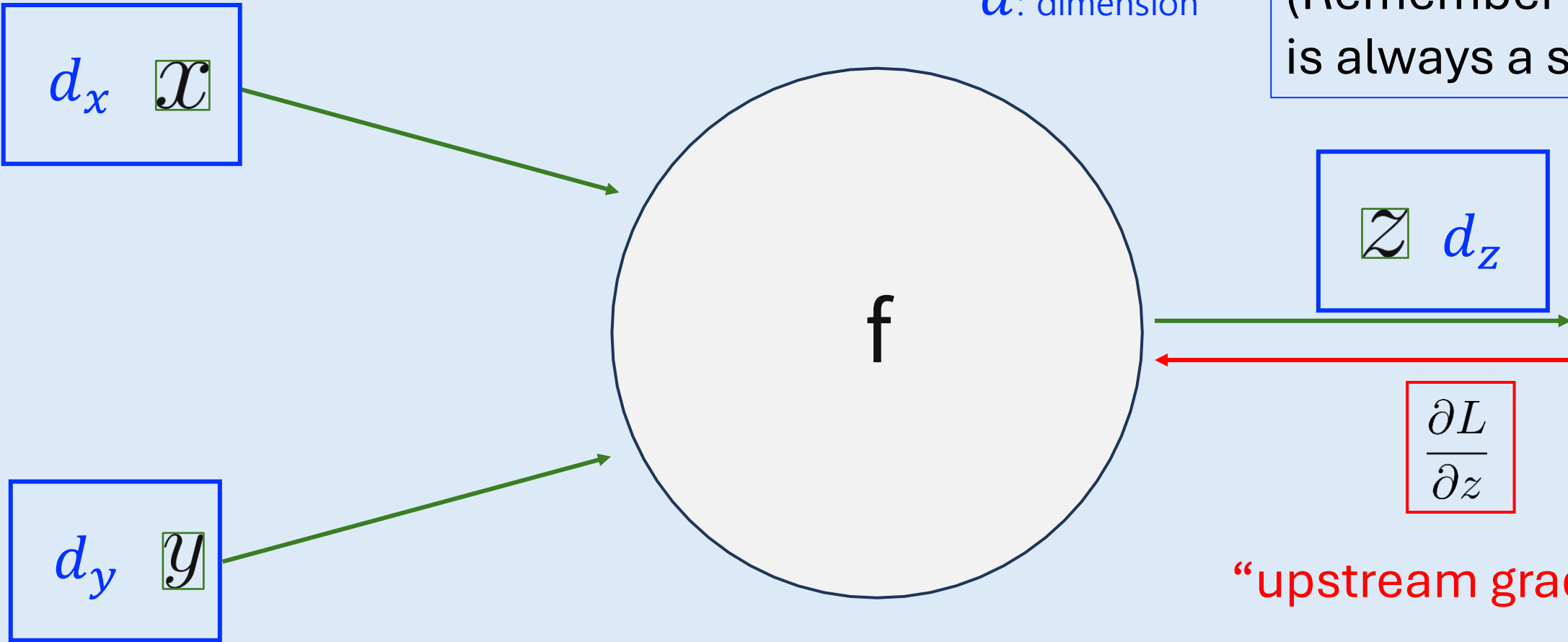
Loss L still a scalar!
(Remember that loss
is always a scalar)



Backprop with vectors

d : dimension

Loss L still a scalar!
(Remember that loss
is always a scalar)

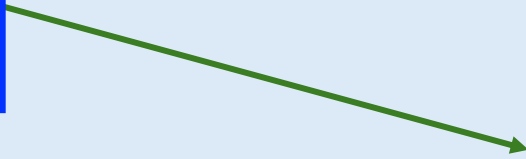


Backprop with vectors

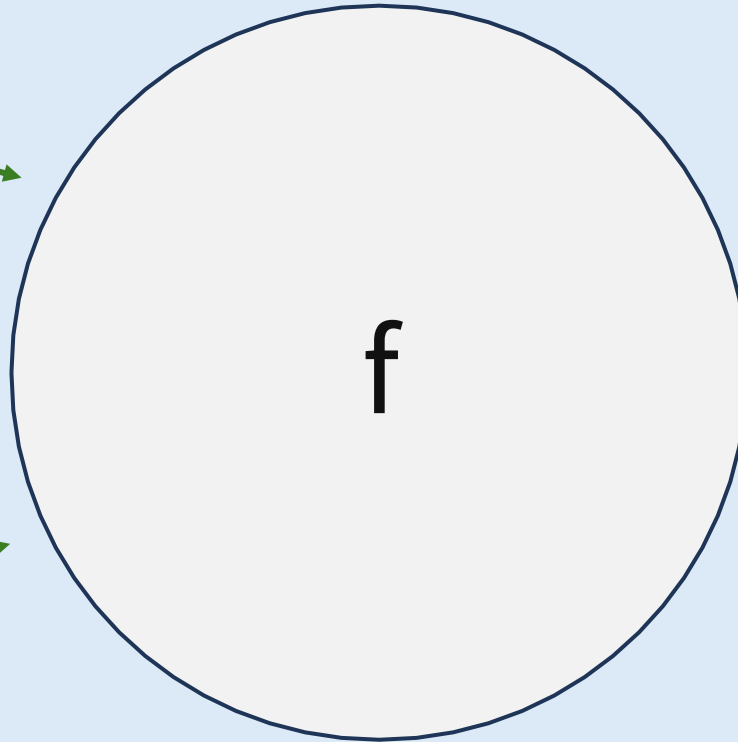
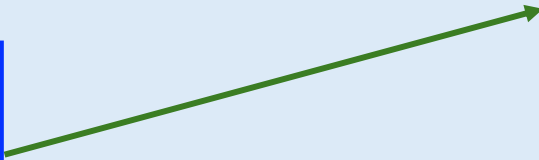
d : dimension

Loss L still a scalar!
(Remember that loss is always a scalar)

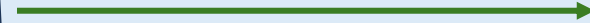
$$d_x \quad \boxed{x}$$



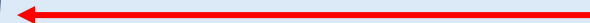
$$d_y \quad \boxed{y}$$



$$\boxed{z} \quad d_z$$



$$\boxed{\frac{\partial L}{\partial z}} \quad d_z$$



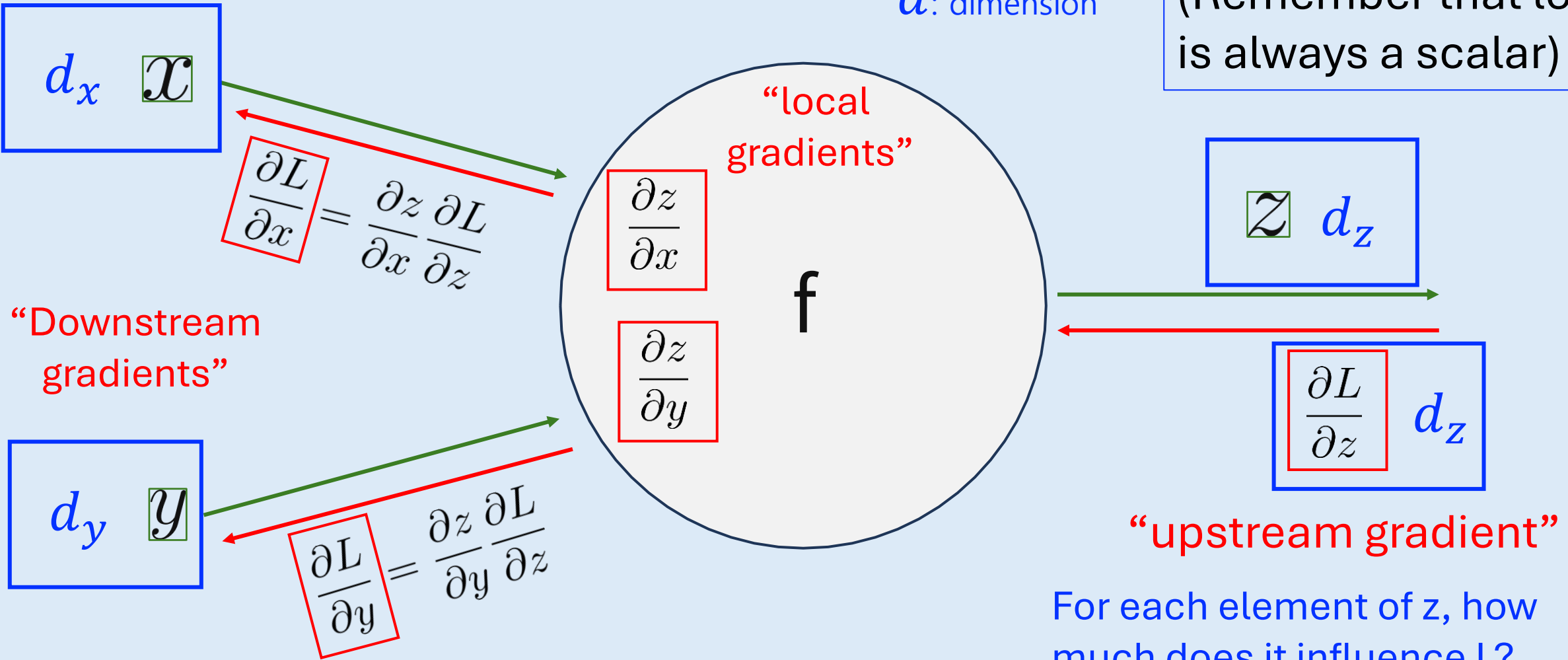
“upstream gradient”

For each element of z , how much does it influence L ?

Backprop with vectors

d : dimension

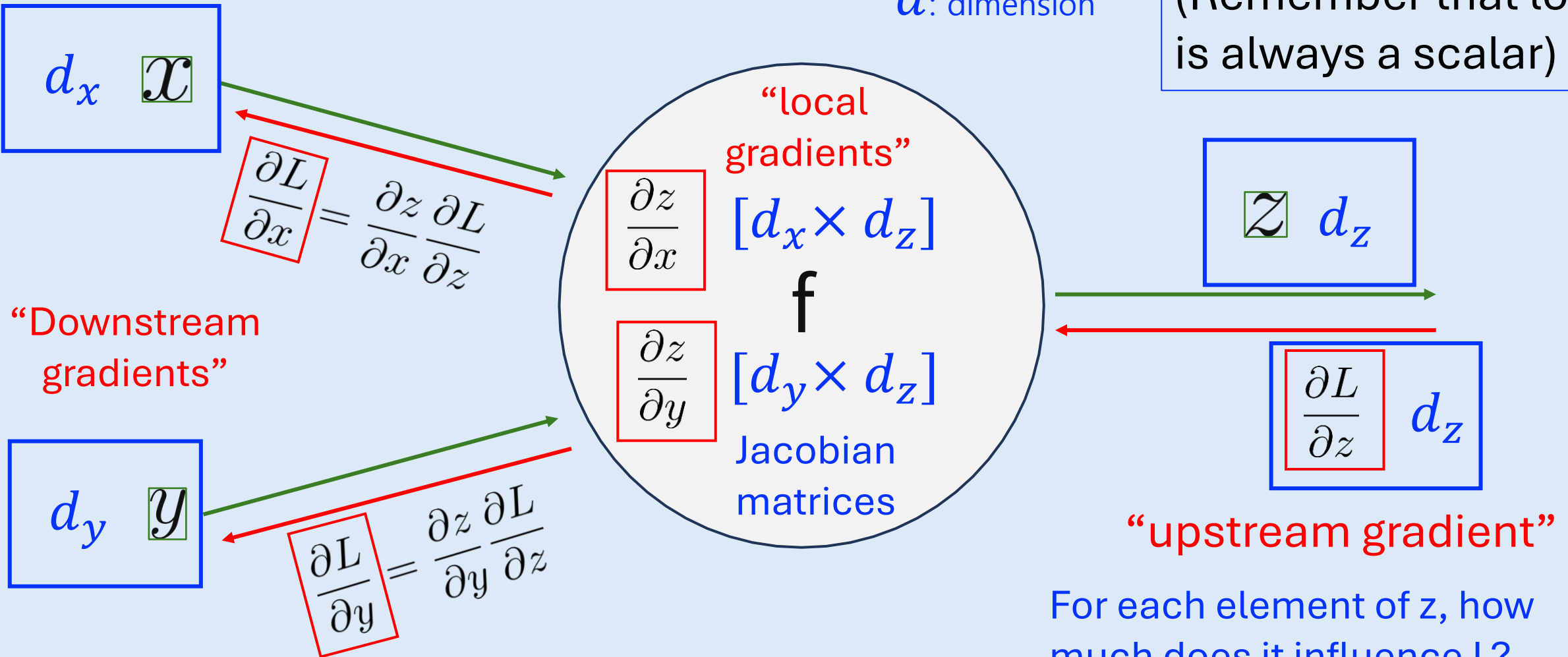
Loss L still a scalar!
(Remember that loss is always a scalar)



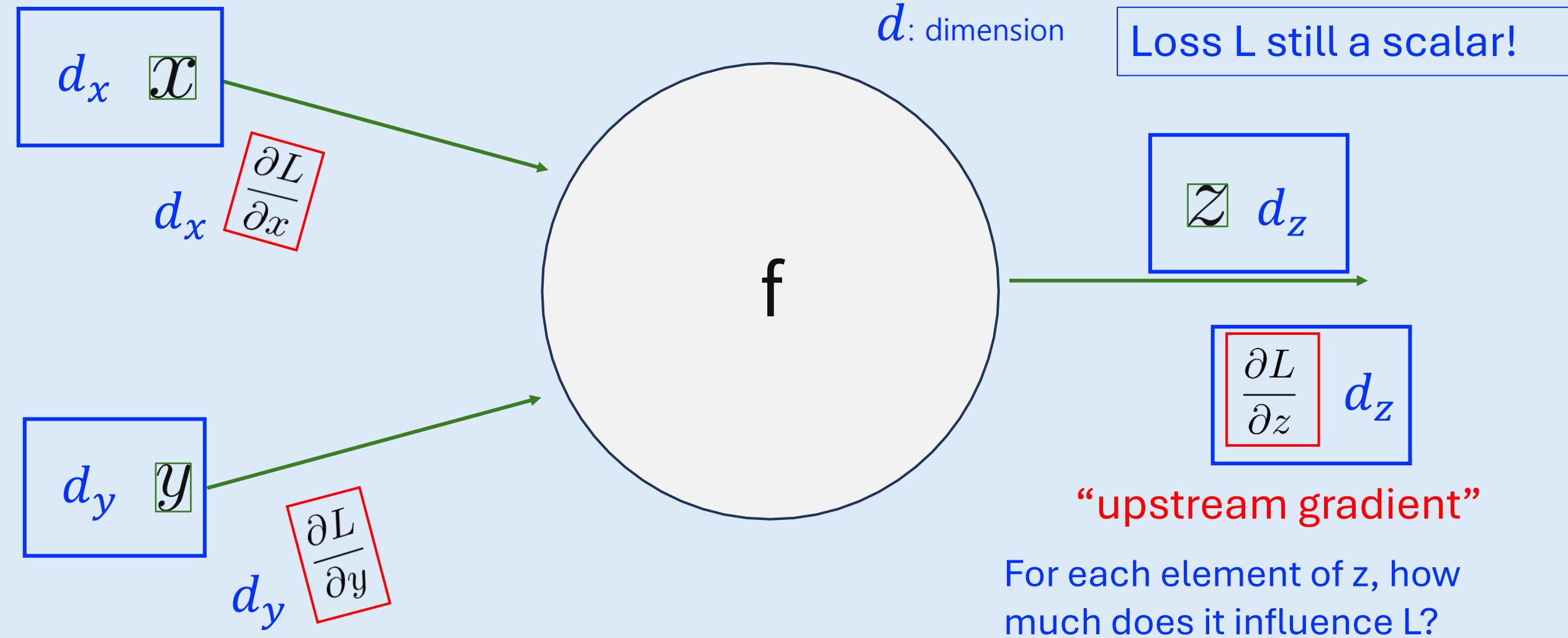
Backprop with vectors

d : dimension

Loss L still a scalar!
(Remember that loss is always a scalar)



Gradients of variables wrt loss have same dims as the original variable



Backprop with vectors

4d input x:

$\begin{bmatrix} 1 \\ -2 \\ 3 \\ -1 \end{bmatrix}$

$f(x) = \max(0, x)$
(elementwise)

4d output z:

$\begin{bmatrix} 1 \\ -2 \\ 3 \\ -1 \end{bmatrix}$

Backprop with vectors

4d input x:

$\begin{bmatrix} 1 \\ -2 \\ 3 \\ -1 \end{bmatrix}$

$$f(x) = \max(0, x) \text{ (elementwise)}$$

4d output z:

$\begin{bmatrix} 1 \\ -2 \\ 3 \\ -1 \end{bmatrix}$

4d dL/dx:

$\begin{bmatrix} 4 \\ 0 \\ 5 \\ 0 \end{bmatrix}$

[dz/dx] [dL/dz]

$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 4 \\ -1 \\ 5 \\ -9 \end{bmatrix}$

4d dL/dz:

$\begin{bmatrix} 4 \\ -1 \\ 5 \\ -9 \end{bmatrix}$

Upstream
gradient

Jacobian is sparse: off-diagonal entries always zero! Never explicitly form Jacobian -- instead use implicit multiplication

Backprop with vectors

4d input x:

$\begin{bmatrix} 1 \\ -2 \\ 3 \\ -1 \end{bmatrix}$

$$f(x) = \max(0, x) \\ \text{(elementwise)}$$

4d output z:

$\begin{bmatrix} 1 \\ -2 \\ 3 \\ -1 \end{bmatrix}$

4d dL/dx :

$\begin{bmatrix} 4 \\ 0 \\ 5 \\ 0 \end{bmatrix}$

$[dz/dx] \quad [dL/dz]$

$$\left(\frac{\partial L}{\partial x} \right)_i = \begin{cases} \left(\frac{\partial L}{\partial z} \right)_i & \text{if } x_i > 0 \\ 0 & \text{otherwise} \end{cases}$$

4d dL/dz :

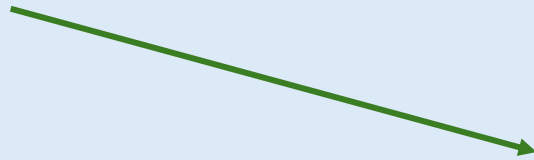
$\begin{bmatrix} 4 \\ -1 \\ 5 \\ -9 \end{bmatrix}$

Upstream
gradient

Jacobian is sparse: off-diagonal entries always zero! Never explicitly form Jacobian -- instead use implicit multiplication

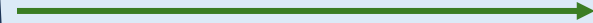
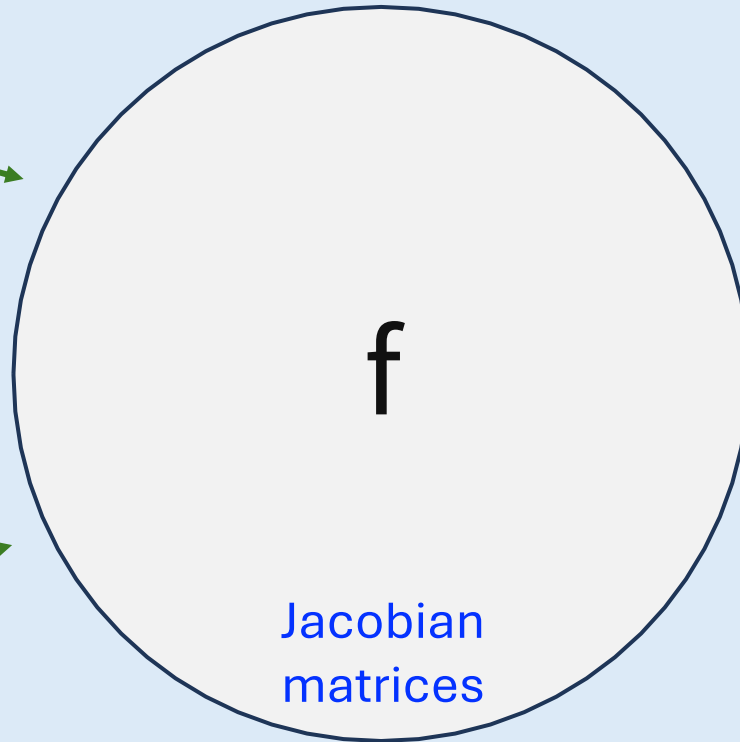
Backprop with matrices (or tensors)

$$[D_x \times M_x]$$

 $\mathcal{X}$ 

Matrix-vector
multiply

$$[D_y \times M_y]$$

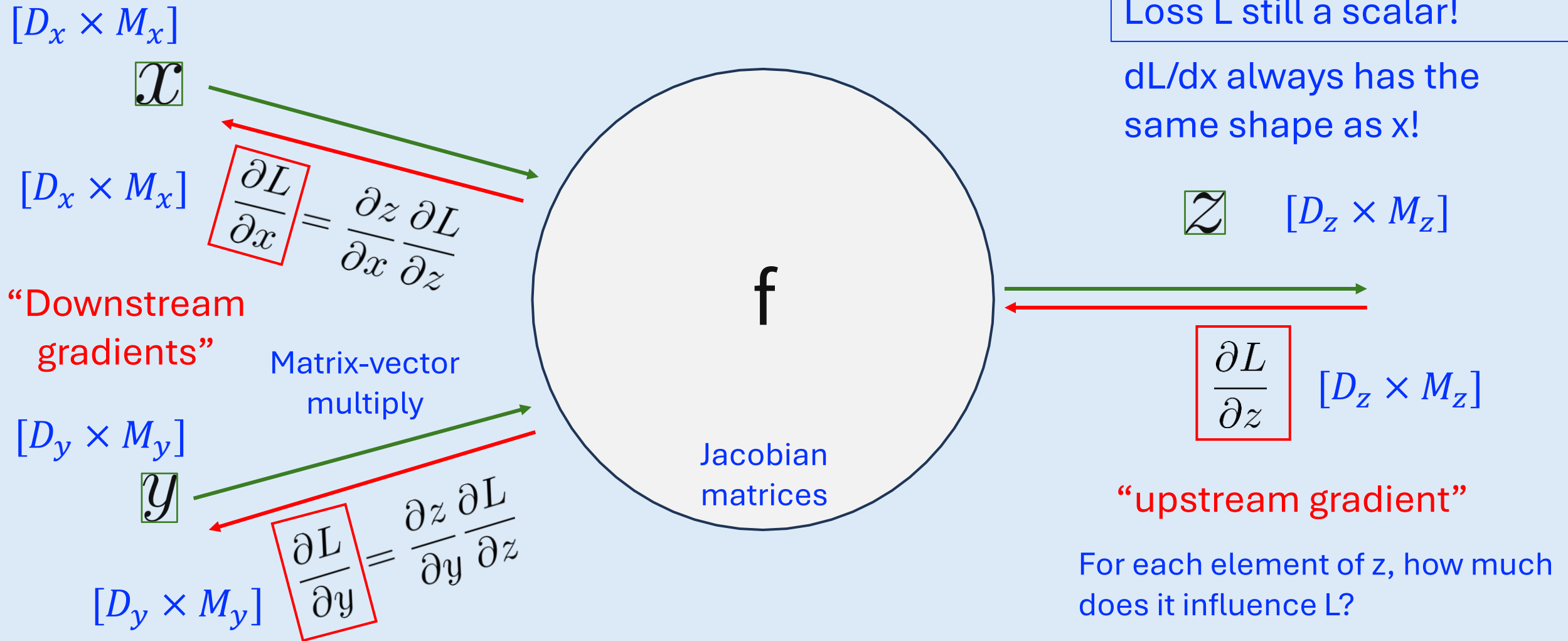
 $\mathcal{Y}$  $\mathcal{Z}$

$$[D_z \times M_z]$$

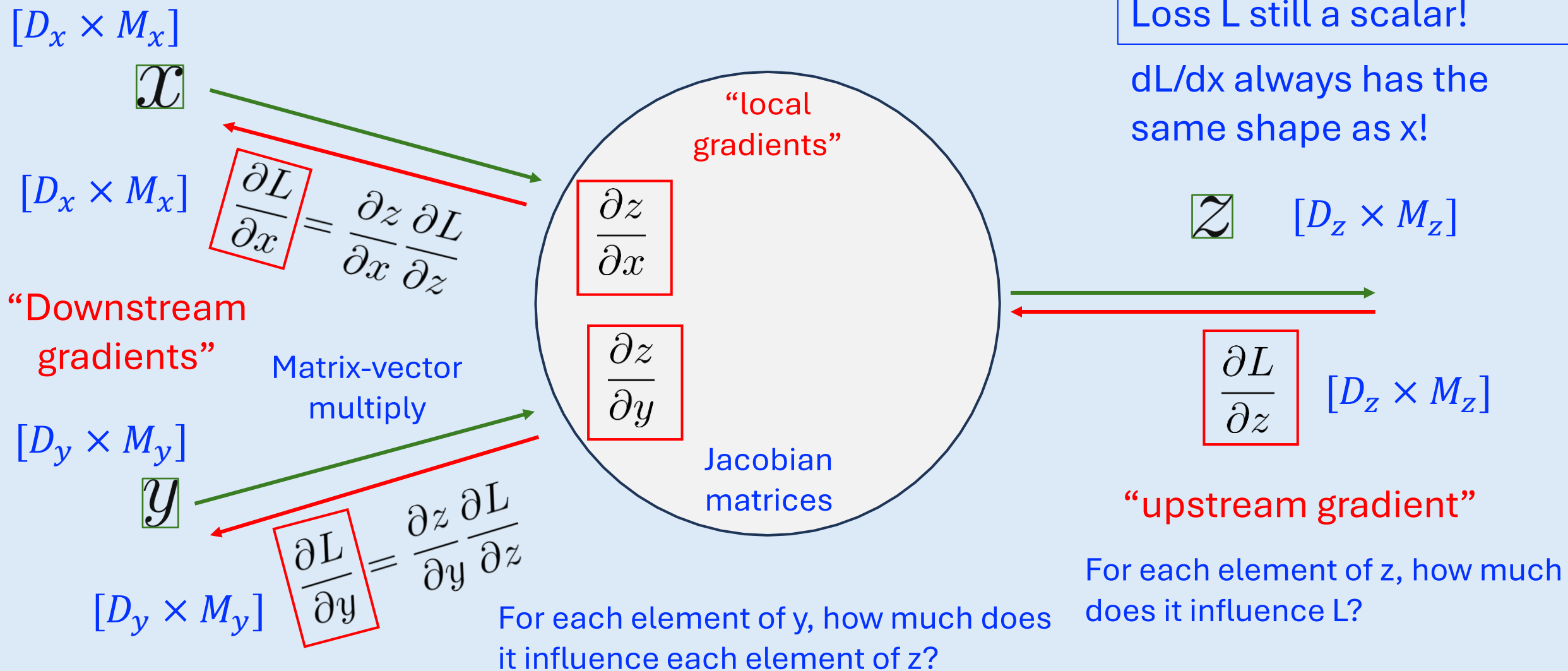
Loss L still a scalar!

dL/dx always has the
same shape as x !

Backprop with matrices (or tensors)



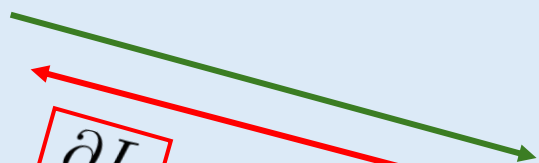
Backprop with matrices (or tensors)



Backprop with matrices (or tensors)

$$[D_x \times M_x]$$

x



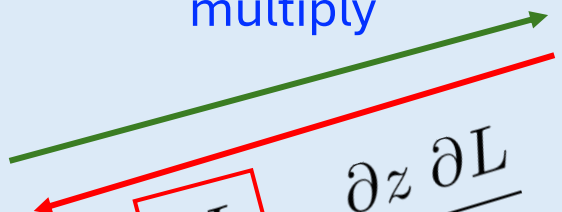
$$\frac{\partial L}{\partial x} = \frac{\partial z}{\partial x} \frac{\partial L}{\partial z}$$

“Downstream
gradients”

Matrix-vector
multiply

$$[D_y \times M_y]$$

y



$$\frac{\partial L}{\partial y} = \frac{\partial z}{\partial y} \frac{\partial L}{\partial z}$$

$$[D_y \times M_y]$$

For each element of y , how much does
it influence each element of z ?

“local
gradients”

$$\frac{\partial z}{\partial x}$$

$$[(D_x \times M_x) \times (D_z \times M_z)]$$

$$\frac{\partial z}{\partial y}$$

$$[(D_y \times M_y) \times (D_z \times M_z)]$$

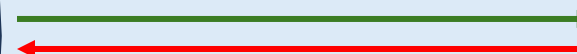
Jacobian
matrices

Loss L still a scalar!

dL/dx always has the
same shape as x !

z

$$[D_z \times M_z]$$



$$\frac{\partial L}{\partial z}$$

$$[D_z \times M_z]$$

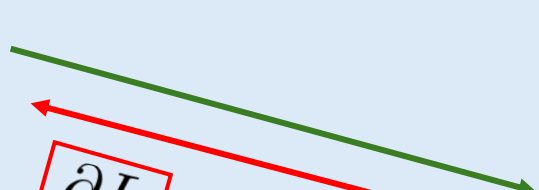
“upstream gradient”

For each element of z , how
much does it influence L ?

Backprop with matrices (or tensors)

$$[D_x \times M_x]$$

x



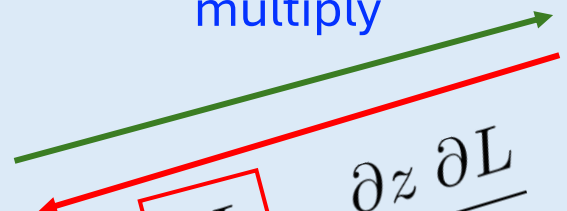
$$\frac{\partial L}{\partial x} = \frac{\partial z}{\partial x} \frac{\partial L}{\partial z}$$

“Downstream
gradients”

Matrix-vector
multiply

$$[D_y \times M_y]$$

y



$$\frac{\partial L}{\partial y} = \frac{\partial z}{\partial y} \frac{\partial L}{\partial z}$$

$$[D_y \times M_y]$$

For each element of y , how much does
it influence each element of z ?

“local
gradients”

$$\frac{\partial z}{\partial x}$$

$$[(D_x \times M_x) \times (D_z \times M_z)]$$

$$\frac{\partial z}{\partial y}$$

$$[(D_y \times M_y) \times (D_z \times M_z)]$$

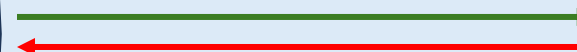
Jacobian
matrices

Loss L still a scalar!

dL/dx always has the
same shape as x !

z

$$[D_z \times M_z]$$



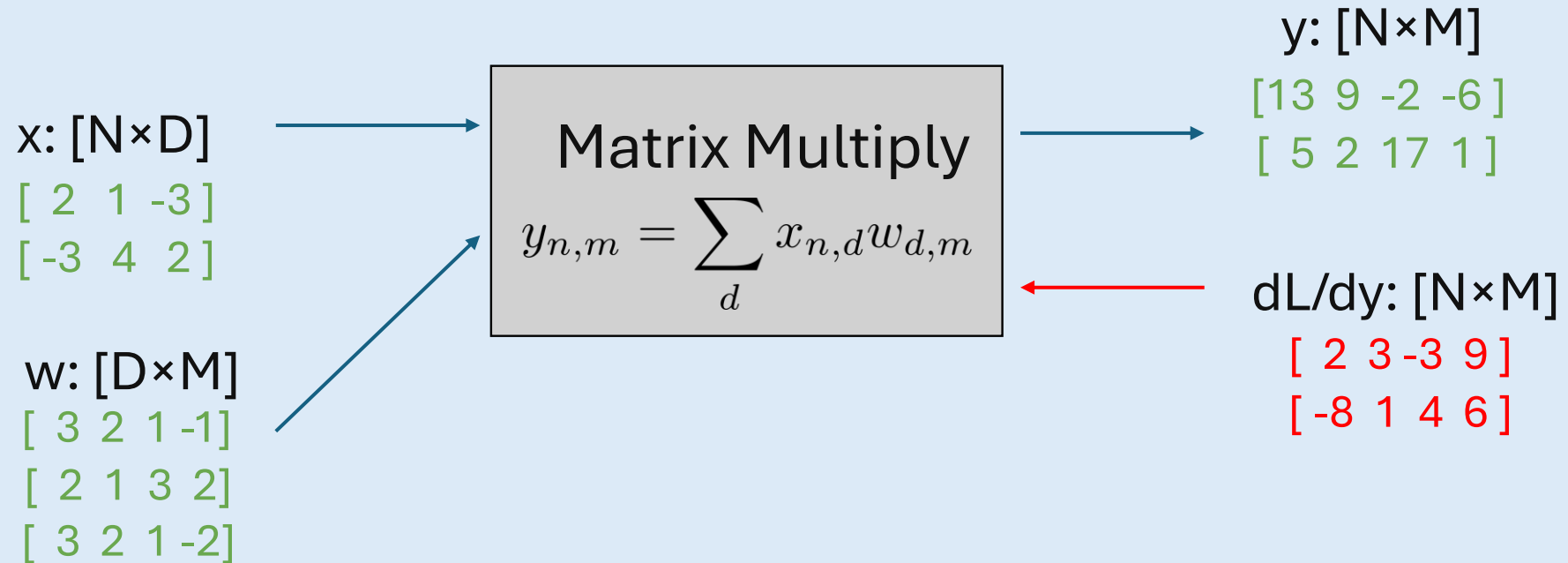
$$\frac{\partial L}{\partial z}$$

$$[D_z \times M_z]$$

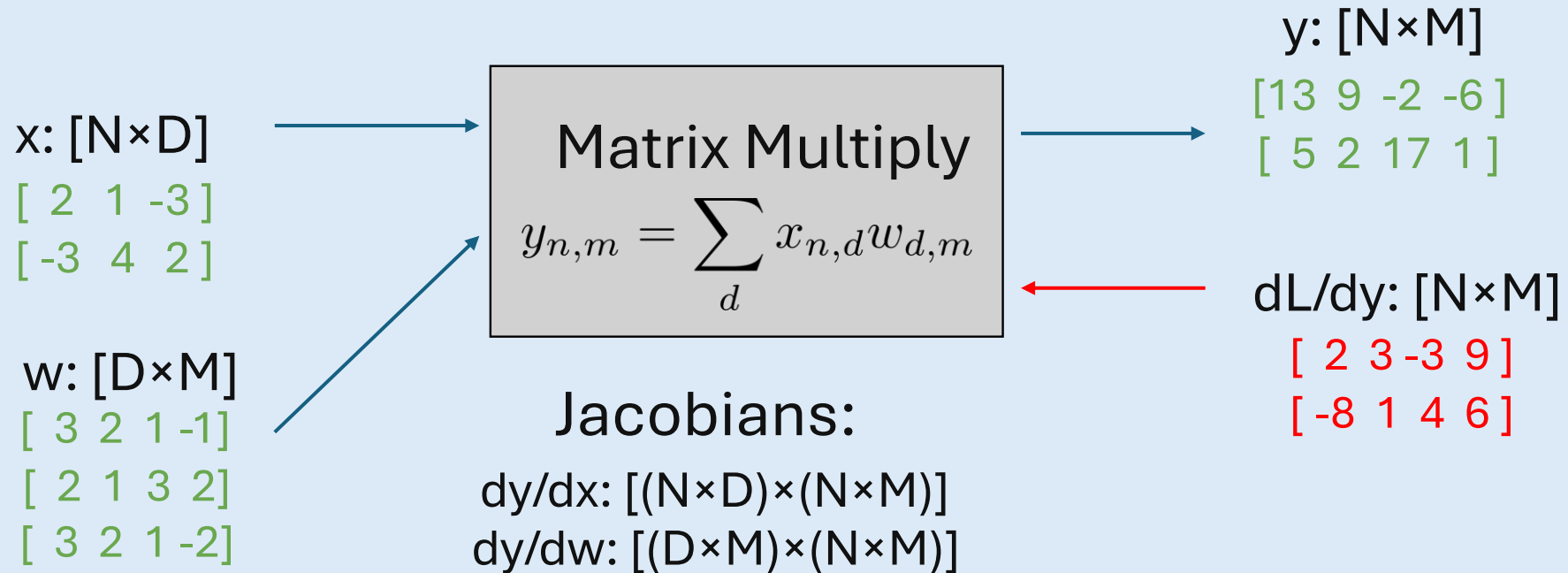
“upstream gradient”

For each element of z , how
much does it influence L ?

Backprop with matrices (or tensors)



Backprop with matrices (or tensors)



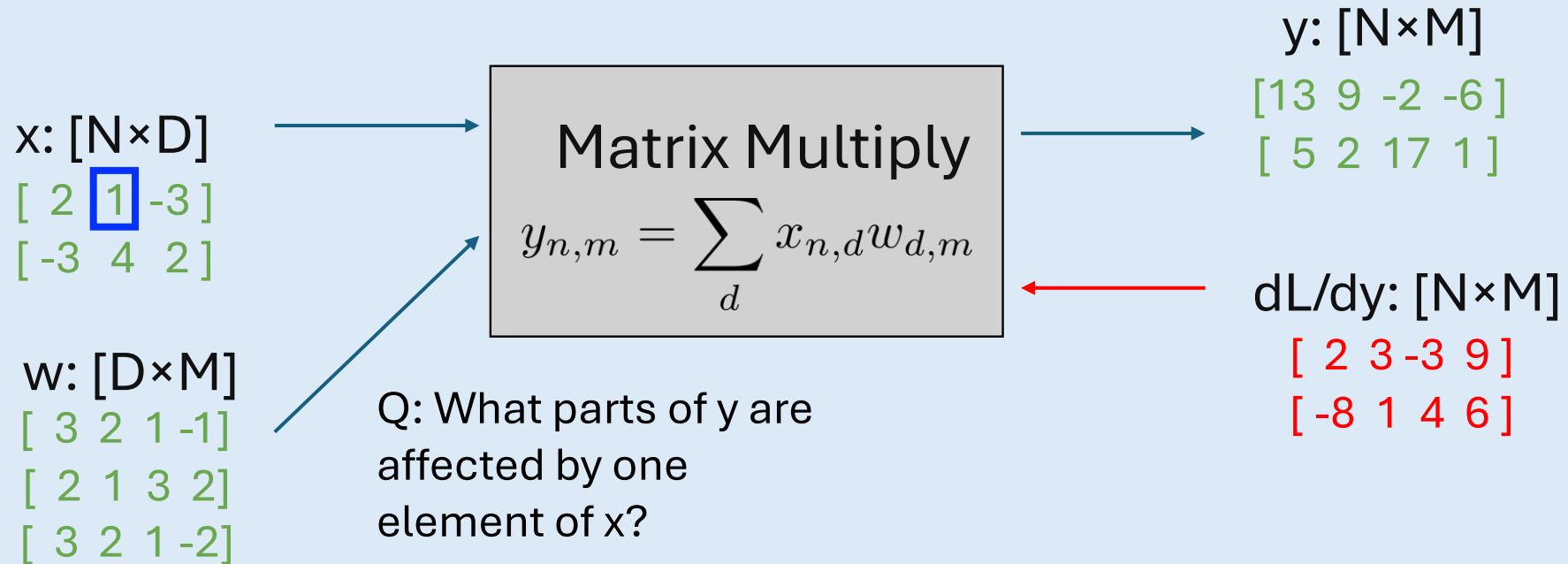
For a neural net we may have

$N=64, D=M=4096$

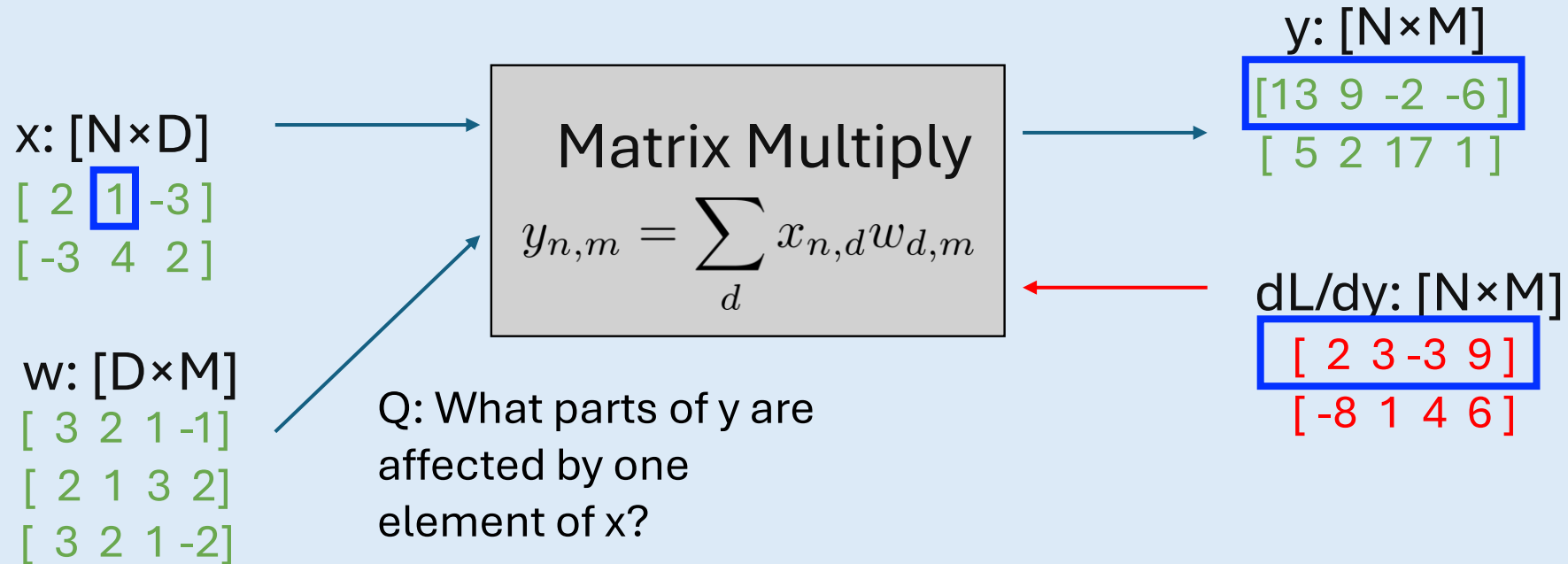
Each Jacobian takes ~256 GB of memory!

Must work with them implicitly!

Backprop with matrices (or tensors)



Backprop with matrices (or tensors)

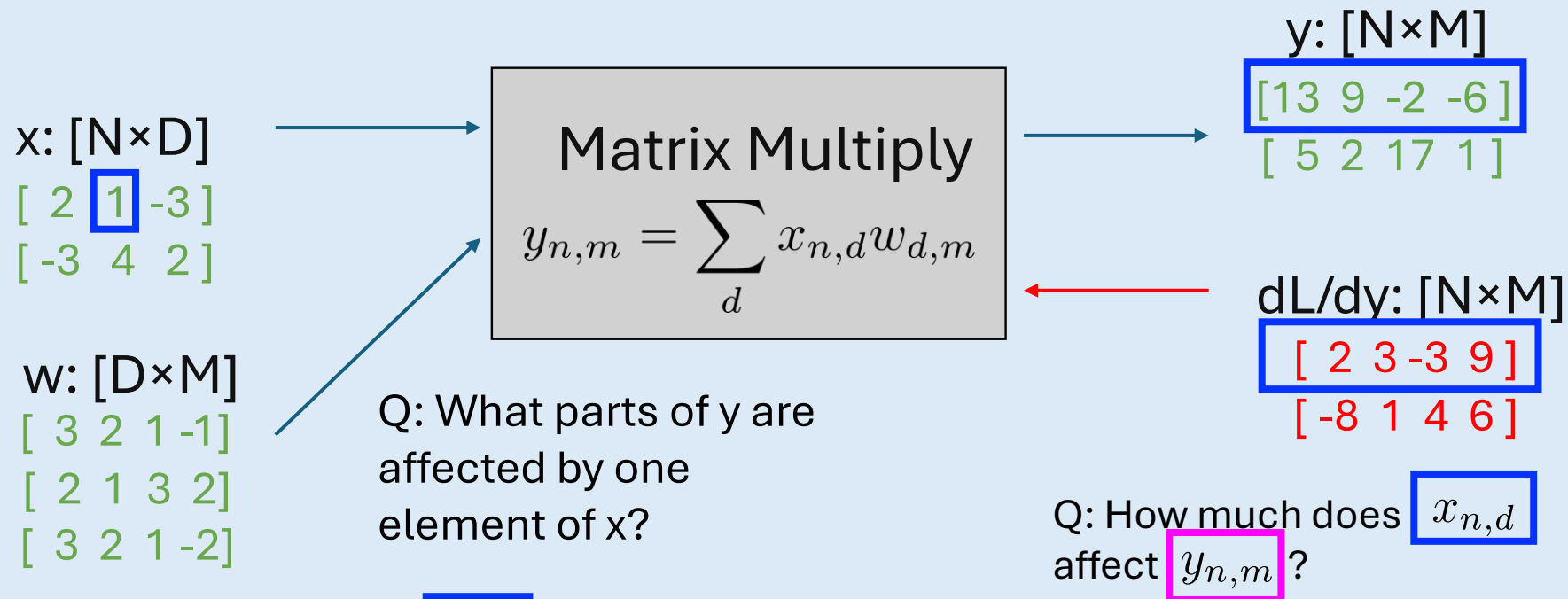


Q: What parts of y are affected by one element of x ?

A: $x_{n,d}$ affects the whole row y_n .

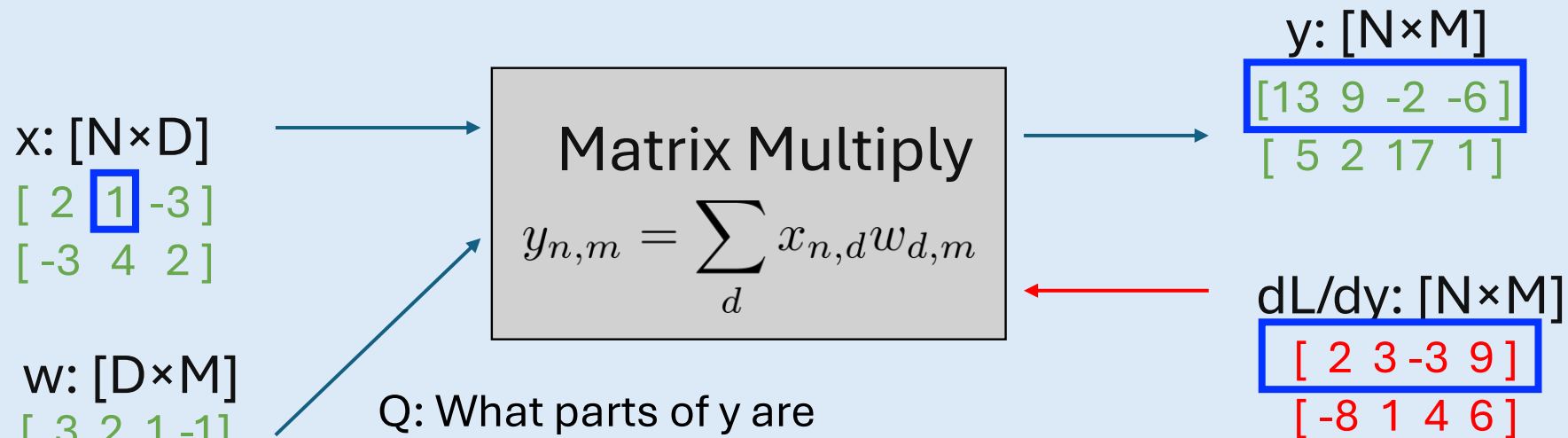
$$\frac{\partial L}{\partial x_{n,d}} = \sum_m \frac{\partial L}{\partial y_{n,m}} \frac{\partial y_{n,m}}{\partial x_{n,d}}$$

Backprop with matrices (or tensors)



$$\frac{\partial L}{\partial x_{n,d}} = \sum_m \frac{\partial L}{\partial y_{n,m}} \frac{\partial y_{n,m}}{\partial x_{n,d}}$$

Backprop with matrices (or tensors)



Q: What parts of y are affected by one element of x ?

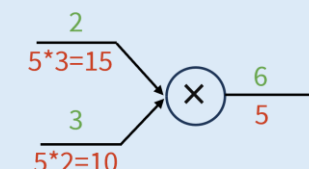
A: $x_{n,d}$ affects the whole row y_n .

$$\frac{\partial L}{\partial x_{n,d}} = \sum_m \frac{\partial L}{\partial y_{n,m}} \frac{\partial y_{n,m}}{\partial x_{n,d}} = \sum_m \frac{\partial L}{\partial y_{n,m}} w_{d,m}$$

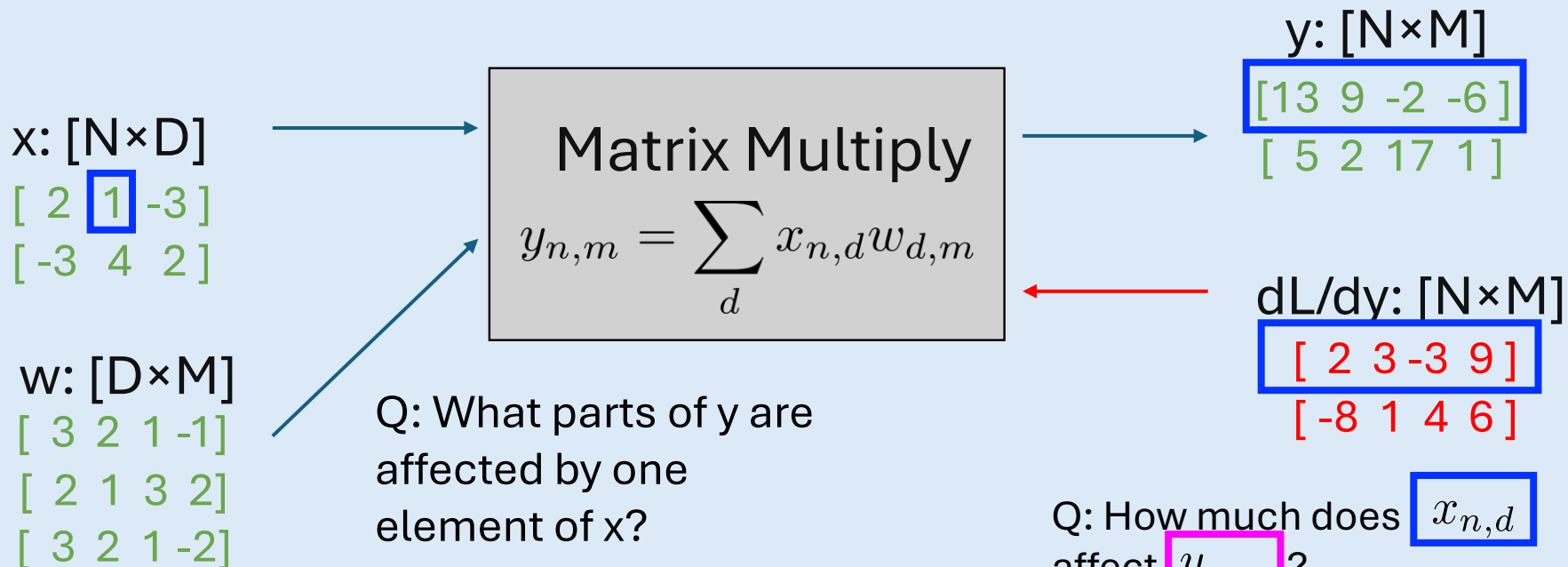
Q: How much does $x_{n,d}$ affect $y_{n,m}$?

A: $w_{d,m}$

mul gate: "swap multiplier"



Backprop with matrices (or tensors)



Q: What parts of y are affected by one element of x ?

A: $x_{n,d}$ affects the whole row y_n .

Q: How much does $x_{n,d}$ affect $y_{n,m}$?

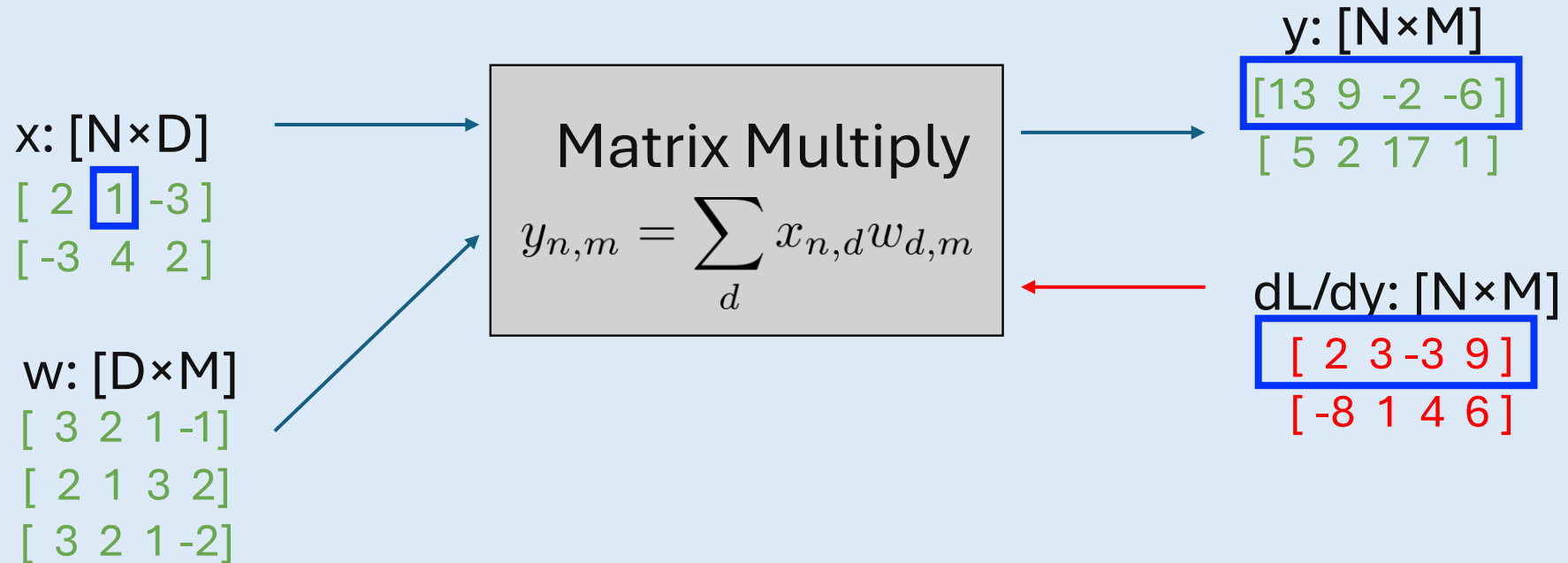
A: $w_{d,m}$

$[N \times D]$ $[N \times M]$ $[M \times D]$

$$\frac{\partial L}{\partial x} = \left(\frac{\partial L}{\partial y} \right) w^T$$

$$\frac{\partial L}{\partial x_{n,d}} = \sum_m \frac{\partial L}{\partial y_{n,m}} \frac{\partial y_{n,m}}{\partial x_{n,d}} = \sum_m \frac{\partial L}{\partial y_{n,m}} w_{d,m}$$

Backprop with matrices (or tensors)



$[N \times D] \quad [N \times M] \quad [M \times D]$

$$\frac{\partial L}{\partial x} = \left(\frac{\partial L}{\partial y} \right) w^T$$

$[D \times M] \quad [D \times N] \quad [N \times M]$

$$\frac{\partial L}{\partial w} = x^T \left(\frac{\partial L}{\partial y} \right)$$

These formulas are easy to remember: they are the only way to make shapes match up!

References

- Stanford CS231n
- Mathematical Foundations of Deep Neural Networks at Seoul National University.

